

DESIGN AND EVALUATION OF A SELF-HOSTED MULTI-CURRENCY PERSONAL FINANCE WEB APPLICATION FOR THE ALBANIAN CONTEXT

By

Krisjor Mema

Dr. (c). Alda Myzeqari



A THESIS

Submitted to

KOLEGJI UNIVERSITAR "Instituti Kanadez i Teknologjise"

University College "CANADIAN INSTITUTE OF TECHNOLOGY"

Faculty of Engineering

Department of Software Engineering

In partial fulfillment of the requirements for the degree of:

Bachelor in Software Engineering

Submitted on September 1st, 2026

Submitted on September 1st, 2026

Approved:

(signature) Dr.(c). Alda Myzeqari

(signature) Head of Department

I am aware that my thesis will be included in the Canadian Institute of Technology's collection. My signature below authorizes the release of my thesis to any reader upon request. I also confirm that the work represented in this thesis is my own.

(Signature) Krisjor Mema, Author

**TITLE OF THESIS: DESIGN AND EVALUATION OF A SELF-HOSTED
MULTI-CURRENCY PERSONAL FINANCE WEB APPLICATION FOR THE
ALBANIAN CONTEXT**

Abstract

Personal finance in Albania is commonly managed through spreadsheets, banking interfaces, or tools designed for other markets. The products reviewed in this thesis either do not provide practical support for Albanian banks or require more technical setup than is desirable for individual self-hosting. This work develops Vault, a self-hosted personal finance web application intended to address that gap. Vault uses a React 18 frontend, a Flask 3 REST API, PostgreSQL 15, and Docker Compose. Its main functions are account and transaction management, categorisation, budgeting, savings goals, reporting, multi-currency storage, and CSV statement import. The importer uses a generic preview-and-mapping workflow rather than bank-specific parser classes and was designed against statement-export samples from four purposively selected major Albanian retail banks. Evaluation uses a seeded ninety-day dataset of roughly 150 transactions, sixteen automated tests, and server-side response-time measurements. All sixteen automated tests passed (100%), including five importer tests and one cross-user isolation test. Ordinary interactive endpoints remained below the 100 ms reference at demonstration scale, with account-related requests completing in under 10 ms and the dashboard summary in approximately 40–80 ms. Login measured approximately 280 ms because of bcrypt password verification, while a 90-row CSV import required approximately 150–250 ms due to row parsing and SHA-256 duplicate checking. The results show that the proposed architecture is workable for the evaluated personal-use setting, while broader usability, security, and scalability testing remains outside the scope of this thesis.

Keywords: personal finance management; web application; REST API; React; Flask; PostgreSQL; multi-currency; budgeting; self-hosted; Albania.

Acknowledgments

I thank my advisor, Dr. (c). Alda Myzeqari, for her guidance throughout this project, and in particular for asking the difficult questions early, when there was still time to do something about the answers.

This project relies on a substantial open-source software ecosystem, including Flask, React, PostgreSQL, SQLAlchemy and many smaller libraries. I am grateful to the developers and maintainers whose work made the implementation possible.

Finally, I thank my family for their patience and support throughout the months devoted to this project.

Table of Contents

Abstract.....	ii
Acknowledgments	iii
Table of Contents.....	iv
List of Figures.....	vii
List of Tables.....	vii
List of Appendices.....	vii
CHAPTER I: INTRODUCTION	1
I.1 Background	1
I.2 Research Problem.....	1
I.3 Research Questions	2
I.4 Objectives.....	2
I.5 Scope and Limitations	3
I.5.1 Scope	3
I.5.2 Limitations	3
I.6 Significance	4
Chapter II: Literature Review	6
II.1 Literature.....	6
II.1.1 Historical Development of PFM Software	6
II.1.2 The Canonical Functional Surface.....	6
II.1.3 Architectural and Security Foundations.....	6
II.1.4 Regional Context and Open Banking	7
II.2 Comparative Analysis of Existing Systems.....	7
II.3 Research Gap	8
Chapter III: Methodology	10
III.1 Research Design	10
III.2 Data and Datasets	11
III.2.1 Secondary Data.....	11
III.2.2 Primary Data - Demonstration Dataset.....	11
III.2.3 Primary Data - Bank Statement Format Samples	12
III.2.4 Reproducibility of the Data	12
III.3 Tools and Technologies.....	13
III.3.1 Justifying the Stack as a Whole	14
III.3.2 Alternatives Considered at the Tool Level	15
III.4 Evaluation Metrics.....	15

III.4.1 Justifying the Metric Choices	16
Chapter IV: System Design and Architecture	17
IV.1 Architectural Overview	17
IV.2 Entity-Relationship Model	18
IV.3 Use Case Overview	19
IV.4 Sequence Diagrams	20
IV.4.1 Authentication Flow	20
IV.4.2 CSV Import Pipeline	21
IV.5 Technology Stack and Design Decisions	22
Chapter V: Implementation	24
V.1 Development Environment Setup	24
V.2 Backend Code Organisation	24
V.3 Database Implementation	25
V.4 API Layer.....	26
V.5 Frontend Implementation.....	28
V.5.1 Dashboard Screen	30
V.5.2 Transactions Screen	30
V.5.3 Budgets and Goals Screens.....	31
V.6 Testing	32
Chapter VI: Evaluation and Results	34
VI.1 Experimental Setup	34
VI.1.1 Hardware and Software Environment	34
VI.1.2 Test Database	34
VI.1.3 Baselines.....	34
VI.2 Results	34
VI.2.1 Functional Coverage.....	35
VI.2.2 Correctness via Automated Testing	36
VI.2.3 Performance Characterisation	37
VI.3 Discussion	39
VI.3.1 Interpretation of the Results	39
VI.3.2 Connection to the Existing Literature	39
VI.3.3 Justifying the Major Design Decisions in Light of Results.....	40
VI.4 Threats to Validity.....	40
VI.4.1 Internal Validity	40
VI.4.2 External Validity	40
VI.4.3 Construct Validity	41

VI.4.4 Reliability	41
Chapter VII: Conclusions and Future Work.....	42
VII.1 Conclusions	42
VII.2 Future Work	43
Bibliography	44
Appendix A: Source Code Repository	48
Appendix B: Demonstration Dataset	49
Appendix C: Detailed Test Results	50
Appendix D: Installation Guide.....	51

List of Figures

Figure 1: Methodology overview	10
Figure 2: System architecture (three-tier).....	17
Figure 3: Entity-relationship diagram.....	18
Figure 4: Use case overview.....	19
Figure 5: Sequence diagram: user login and JWT issuance	20
Figure 6: Sequence diagram: CSV import pipeline.....	21
Figure 7: Application-factory pattern in the Flask backend	25
Figure 8: React component hierarchy.....	29
Figure 9: Dashboard screen	30
Figure 10: Transactions screen with date grouping.....	31
Figure 11: Budgets screen with progress indicators.....	32
Figure 12: Automated test results by file.....	37

List of Tables

Table 1: Comparative analysis of existing personal finance management systems.....	7
Table 2: Tools and technologies used in the implementation	13
Table 3: Evaluation metrics by category	15
Table 4: Design decisions, alternatives considered, and chosen approaches	22
Table 5: Database tables and their cardinality on the demonstration dataset.....	26
Table 6: API endpoint surface by resource	27
Table 7: Feature-to-objective traceability matrix	35
Table 8: Automated test results	36
Table 9: Endpoint response-time characterisation (median of 7 warm requests).....	37

List of Appendices

Appendix A: Source Code Repository	48
Appendix B: Demonstration Dataset.....	49
Appendix C: Detailed Test Results	50
Appendix D: Installation Guide.....	51

CHAPTER I: INTRODUCTION

I.1 Background

Personal financial management is often informal. Studies of everyday money practices report the use of spreadsheets, paper notes, mental calculation and banking apps rather than dedicated PFM software (Kaye et al., 2014). Even widely known PFM products serve only part of the potential market; when Intuit announced Mint's closure in late 2023, the service had roughly 3.6 million monthly active users (Bloomberg News, 2023).

Fragmented records make routine financial questions harder to answer. A user may want to know how much was spent on groceries last month, whether a savings goal is on schedule, which subscriptions are recurring, or how the current month compares with the previous one. Answering these questions consistently requires a consolidated transaction history rather than separate notes and account views. Research also links financial literacy with saving, borrowing and long-term household outcomes, while distinguishing financial knowledge from day-to-day behaviour (Mandell & Klein, 2009), (Lusardi & Mitchell, 2014). A practical tracking tool can help reduce the effort required to apply that knowledge in ordinary decisions.

PFM software has a long history. Quicken appeared in 1983 and Microsoft Money in 1991 as desktop applications built around local files. Mint moved the model to the web in 2007 and was acquired by Intuit two years later (Intuit Inc., 2009). More recent products include subscription cloud services such as You Need A Budget and Monarch Money, alongside self-hosted open-source systems such as Firefly III (van Pelt, 2016). Household-finance research treats consumption, saving, borrowing and asset allocation as connected decisions (Campbell, 2006), while the mental-accounting literature helps explain why users organise money into categories and budgets (Thaler, 1999). For this thesis, the practical case is an Albanian user who may hold accounts in lek and foreign currencies and who wants financial records to remain under their own control.

I.2 Research Problem

This thesis addresses the absence, within the reviewed systems, of a self-hostable and multi-currency personal finance application that combines standard PFM functions with an Albanian-context orientation and can be operated without extensive system-administration effort.

In practice, an Albanian user choosing a finance-tracking approach faces trade-offs. Commercial cloud services may not support local banks and require third-party processing of financial data. Self-hosted alternatives such as Firefly III are capable but require deployment and maintenance effort (van Pelt, 2016). Spreadsheets are flexible, but budgeting, categorisation and reporting must be assembled manually. The gap addressed here is therefore not the total absence of these capabilities, but the lack of a reviewed option that combines local statement handling, multi-currency storage, self-hosting and relatively low operational complexity in one personal-use system.

Leaving this gap unresolved can limit practical choice for local users. Cloud products introduce third-party data-processing and possible cross-border privacy considerations (European Parliament and Council of the European Union, 2016), while self-hosted alternatives shift more operational responsibility to the user. The project therefore investigates whether a locally oriented self-hosted design can reduce those trade-offs within a clearly bounded scope.

I.3 Research Questions

Three research questions guide the work:

1. Which architecture and data model are suitable for a self-hostable, multi-currency personal finance application aimed at the Albanian context?
2. Can a generic, user-mapped CSV importer accommodate representative statement exports from different Albanian banks without requiring a separate parser for every bank?
3. What authentication and per-user isolation scheme is appropriate for a self-hosted PFM application within the scope of the evaluation, without introducing operational complexity the use case does not justify?

I.4 Objectives

The overall objective is to design, build and evaluate a web-based personal finance application for the problem defined above. Six specific objectives are used to guide the implementation and later evaluation:

1. To design and implement a predominantly normalised relational data model covering the seven core entities of personal finance - users, accounts, categories, tags, transactions, budgets and goals - with one deliberate balance denormalisation for interactive reads.
2. To implement a REST API covering the core application resources, with create-read-update-delete operations where appropriate, stateless JSON Web Token (JWT)

authentication, bcrypt password hashing, and per-user ownership filtering enforced on protected data access.

3. To implement analytical reporting for spending by category, monthly trends, top merchants and net worth, performing the principal aggregations in the database and targeting server-side response times below one hundred milliseconds for ordinary interactive endpoints at demonstration scale.
4. To implement a single-page application (SPA) frontend in React with multi-currency support and a clean, readable visual design.
5. To implement a generic CSV import pipeline designed against statement-export samples from four purposively selected major Albanian retail banks, without per-bank parser classes and with SHA-256 deduplication.
6. To validate the result with a sixteen-case automated test suite covering authentication, transaction integrity, CSV importer behaviour and cross-user data isolation.

I.5 Scope and Limitations

I.5.1 Scope

The system is designed for individual, self-hosted personal use. Within scope:

1. A complete three-tier web application: React SPA in the browser, Flask REST API, PostgreSQL database.
2. Multi-currency support for the Albanian lek (ALL), euro (EUR), US dollar (USD), British pound (GBP) and Swiss franc (CHF).
3. Six account types: checking, savings, credit card, cash, investment, loan.
4. Three transaction types: income, expense, and transfer, the last implemented as a linked pair of records on the two accounts involved.
5. A generic CSV importer that handles comma- and semicolon-delimited files, the common European and ISO date spellings (DD/MM/YYYY, DD.MM.YYYY, DD-MM-YYYY, YYYY-MM-DD, plus written-out forms like “15 Jan 2025”), and amounts in both European and US conventions, including accounting-style parentheses for negatives.

I.5.2 Limitations

The following areas are outside the scope of the implemented and evaluated system:

1. Direct bank API integration. Albania has transposed the EU's PSD2 open-banking framework (European Parliament and Council of the European Union, 2015) into national

law, but the resulting interfaces are reserved for providers licensed by the central bank, which a self-hosted personal application cannot be; data still enters the system through user-initiated CSV import. Section II.1.4 returns to this.

2. Automatic exchange-rate lookup. Currency is stored per account and transaction, but no exchange-rate feed or automatic conversion layer is implemented. Income/expense totals preserve per-currency buckets and net-worth reporting uses the user's default currency; category and time-series analyses should therefore be interpreted within a common reporting currency.
3. Security hardening beyond the evaluated scope. Stateless JWT logout relies on client-side token removal and no server-side token revocation list is implemented. Production TLS termination, secret rotation, penetration testing, dependency auditing and broader adversarial testing are outside the evaluated scope.
4. Receipt photos and optical character recognition.
5. Investment portfolio tracking with market data.
6. Tax preparation and any form of regulatory certification.
7. Native mobile apps. The web frontend is responsive, but there is no separate iOS or Android client.
8. Shared budgets for couples or households.

I.6 Significance

Practical contribution. Vault is a working documented application intended for individual deployment and future extension. Its default currency is lek, the seeded categories reflect common local spending patterns, and the CSV workflow was designed against statement exports from four purposively selected major Albanian retail banks. The project also sits within a wider shift toward digital financial services (World Bank, 2022) and financial-capability initiatives (OECD, 2020). Within the documentation reviewed for this thesis, no directly comparable open-source system with the same stated orientation was identified.

Academic contribution. The thesis documents the integration of Flask, React, PostgreSQL, SQLAlchemy, JWT authentication and REST design in an undergraduate software-engineering project. It records both the implementation and the reasoning behind the main technical choices, providing a concrete case study that can be examined or extended by other students and developers.

Connection to the degree. The project integrates material from across the curriculum, including database design, web architecture, software engineering practice, security and interface design. It therefore serves as a practical synthesis of several areas of the Computer Science program.

Chapter II: Literature Review

II.1 Literature

The literature review focuses on three areas that directly affect the design of Vault: the development and functional scope of PFM software, the architectural and security principles used by the application, and the Albanian banking context that determines how account data can be obtained. These sources are then used to compare representative systems and define the gap addressed by the project (Garman & Forgue, 2018).

II.1.1 Historical Development of PFM Software

PFM software has gone through three broad generations. The first was desktop: Quicken (1983) and Microsoft Money (1991), single-user programs over local files. Their feature set - accounts, transactions, categories and reports - helped define the genre, and later generations largely inherited it. The second generation moved to the web, with Mint as a prominent example. The third generation diversified into subscription cloud products and self-hosted open-source systems such as Firefly III; Mint itself was discontinued in early 2024 (Garman & Forgue, 2018; Intuit Inc., 2009; van Pelt, 2016-2026; The Verge, 2023).

II.1.2 The Canonical Functional Surface

Across these generations, the functional core has remained relatively stable. Personal-finance texts emphasise account tracking, transaction recording, categorisation, budgeting and reporting, and these functions recur across the systems compared in Section II.2. That stability provides a practical baseline for the artifact developed here. Vault implements those five functions and additionally provides savings goals and CSV statement import (Garman & Forgue, 2018).

II.1.3 Architectural and Security Foundations

The architectural basis is the three-tier model: presentation, application logic and data are separated into distinct responsibilities. This separation supports clearer technology boundaries. For the API, REST provides a model based on a uniform interface, statelessness and client-server separation, while the relational model and normalisation practice provide the data foundation for reducing redundancy and update anomalies (Eckerson, 1995; Fielding, 2000; Codd, 1970; Bass et al., 2021; Fowler, 2002).

For security, the stack is based on established components and guidance. JSON Web Tokens carry signed authentication claims, while RFC 8725 documents deployment practices and

known JWT pitfalls. Passwords are stored with bcrypt. OWASP currently prefers Argon2id for new password-storage implementations; bcrypt remains a legacy/fallback option where Argon2id or scrypt are unavailable. The implementation therefore uses bcrypt as a pragmatic project-level choice rather than a claim of universal best practice. Broader web-application, authentication and REST guidance was also considered (Jones et al., 2015; Jones et al., 2020; Provos & Mazières, 1999; OWASP Foundation, 2021; Temoshok et al., 2025).

II.1.4 Regional Context and Open Banking

One regulatory fact shapes the data-acquisition design. PSD2 established a regulated open-banking framework in the European Union. Albania is not an EU member, but it transposed relevant payment-services provisions through Law No. 55/2020 “On payment services”. In November 2024, the Bank of Albania granted the country’s first open-banking licence. Access operates through the regulated provider framework, so this self-hosted application does not itself act as a licensed third-party provider; user-initiated statement import therefore remains the practical data path (European Parliament & Council of the European Union, 2015; Republic of Albania, 2020; Bank of Albania, 2024).

II.2 Comparative Analysis of Existing Systems

Table **Error! Reference source not found.** compares four representative systems against the requirements of this project. The review uses official documentation, feature and pricing pages, installation guidance and, where source code is available, the public repository. The selected systems provide contrasting examples across deployment model, bookkeeping approach, multi-currency support and bank-data import. (YNAB LLC, n.d.; van Pelt, 2016-2026; GnuCash Project, 1998-2024).

Table 1: Comparative analysis of existing personal finance management systems

System	Model	Multi-currency	Bank integration	Cost	Albanian fit
Mint	Cloud SaaS	Limited	Discontinued	Discontinued 2024	No
YNAB	Cloud SaaS	Yes	Direct import through supported providers/markets; file-based import also available	USD 14.99 / month	Partial

System	Model	Multi-currency	Bank integration	Cost	Albanian fit
Firefly III	Self-hosted PHP	Yes	Importer plugins	Free, open-source	No specific local orientation
GnuCash	Desktop, double-entry	Yes	Manual / OFX	Free, open-source	No specific local orientation
Vault (this work)	Self-hosted Python/JS	Yes	Generic CSV importer	Free, open-source	Designed for Albania

The comparison highlights several differences. Mint was discontinued in 2024. YNAB supports direct import through market-dependent providers, but its documented European coverage does not list Albania; file-based import is also available. Firefly III is the closest self-hosted comparison, although its deployment requires configuration of Docker, environment files, the database and related infrastructure. That effort is relevant to perceived ease of use and effort expectancy. GnuCash uses double-entry bookkeeping, historically associated with Pacioli, which offers stronger accounting structure than the narrower personal-spending model required for Vault (The Verge, 2023; YNAB LLC, n.d.; van Pelt, 2016-2026; Davis, 1989; Venkatesh et al., 2003; GnuCash Project, 1998-2024; Pacioli, 1494).

II.3 Research Gap

The comparison identifies four gaps that are relevant to this project (YNAB LLC, n.d.).

Regional availability. In the reviewed systems, direct bank integration does not provide practical coverage for Albanian banks. YNAB’s documented market coverage does not list Albania, while Albania’s developing open-banking framework operates through regulated providers. For the artifact evaluated here, CSV import is therefore treated as the primary data path rather than a fallback (YNAB LLC, n.d.; Republic of Albania, 2020; Bank of Albania, 2024).

Multi-currency semantics. The compared systems use different approaches to multi-currency support. Vault stores account and transaction currency explicitly and avoids automatic FX

conversion. Income/expense totals retain per-currency buckets and net-worth reporting is restricted to the user's default currency; category and time-series views therefore require a common reporting-currency interpretation (YNAB LLC, n.d.; van Pelt, 2016-2026).

Operational complexity. The self-hosted segment assumes a technical operator. Firefly III is the clearest example in the comparison. The practical effort required to configure and maintain a self-hosted deployment is relevant to perceived ease of use and effort expectancy. This project therefore treats deployment simplicity as a design consideration rather than a claim about actual adoption behaviour (van Pelt, 2016-2026; Davis, 1989; Venkatesh et al., 2003).

Localisation. The reviewed products are general-purpose systems rather than tools designed specifically for Albania. YNAB's documented European import coverage does not list Albania, while Firefly III and GnuCash do not state an Albania-specific orientation. Vault therefore adapts defaults, categories and statement-handling assumptions to the local context; full interface translation remains future work (YNAB LLC, n.d.; van Pelt, 2016-2026; GnuCash Project, 1998-2024).

Vault is positioned as a self-hosted PFM tool that preserves native currency values, uses defaults adapted to the Albanian context and does not depend on direct bank integration (Bank of Albania, 2024).

Chapter III: Methodology

The methodology covers four elements: the research design, the sources and datasets used during development, the technology choices, and the criteria used to evaluate the completed application.

III.1 Research Design

The project uses an applied, design-based research approach. Its main output is the Vault application, which is evaluated against the objectives defined in Chapter I.4. This follows the design-science tradition described by Hevner et al. (2004). The six activities identified by Peffers et al. (2007) closely match this work: problem identification (Chapter I), objective definition (I.4), design and development (Chapters IV and V), demonstration (Chapter V), evaluation (Chapter VI), and communication through the thesis.

Methodologically, the study is applied and constructive, addressing a specific local problem through software development. Design decisions are discussed qualitatively, while Chapter VI provides quantitative evidence through automated tests and response-time measurements. The evaluation is cross-sectional and focuses on one implemented system as of 2026; it does not claim statistical generalisation across users, products, or alternative architectures. Figure 1 summarises the methodology, showing how the design-science approach connects problem identification, objectives, system development, data inputs, evaluation, and thesis communication.

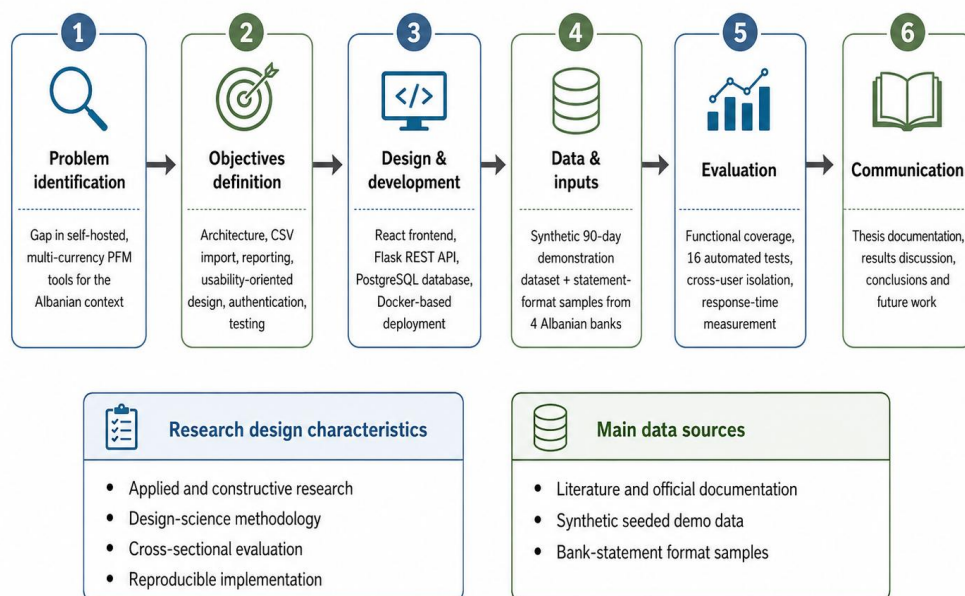


Figure 1: Methodology overview

As shown in Figure 1 the software artifact is both the main development output and the object evaluated in Chapter VI.

Two alternative research designs were considered and set aside. A survey of Albanian users could have quantified finance-tracking habits more precisely, but it would have described the problem rather than produced and evaluated a software artifact. A pure comparative study of existing tools would have had the same limitation. Because the research objective required a working system, a constructive design-science approach was selected.

III.2 Data and Datasets

III.2.1 Secondary Data

Secondary sources include the architecture, REST and relational-design literature cited in Chapter II; official documentation for the four comparison systems (Intuit Inc., 2009), (van Pelt, 2016), (YNAB LLC, n.d.), (GnuCash Project, 2024); the CHI study of personal money-tracking practices (Kaye et al., 2014); the EU open-banking framework (European Parliament and Council of the European Union, 2015); Albania's payment-services law (Republic of Albania, 2020); and specifications or guidance for technologies used in the implementation (Fielding, 2000), (Jones et al., 2015)-(OWASP Foundation, n.d.).

The comparison in Table **Error! Reference source not found.** was carried out using the same review criteria for each system. Official feature pages were used for functionality, pricing pages for cost and currency support, installation guidance for operational effort, and public repositories where available to confirm implementation details. Fixing these criteria before comparison reduced the risk of judging each product by different standards.

III.2.2 Primary Data - Demonstration Dataset

The primary dataset is synthetic and generated by a seeded routine inside the application. It spans ninety days across four accounts: a BKT checking account, Raiffeisen savings, a cash wallet in lek, and a euro-denominated Revolut card. The account model therefore demonstrates storage and display of a second currency, although the seeded transaction generator records its demonstration transactions in ALL. The dataset contains sixteen categories (twelve expense, four income) and around 150 transactions. The exact count can move by a few between runs because the ninety-day window is anchored to the day the seeder executes, but the pseudo-random seed is fixed, so a given execution date produces the same data. A salary of 75,000 ALL lands on the first of every month, rent goes out on the fifth, utilities around the twelfth,

and freelance income arrives every seventeen days. Five monthly budgets (Groceries 25,000 ALL, Restaurants 12,000, Transport 8,000, Entertainment 6,000, Shopping 15,000) and three savings goals (Emergency fund, New laptop, Summer trip) complete the demonstration data. Synthetic data was chosen instead of real bank records for privacy and reproducibility. Transaction histories are highly sensitive personal data, and collecting real records was unnecessary for the functional evaluation. The seeded dataset can be regenerated by another reader, while still providing realistic enough values to demonstrate the application’s behaviour. The objective is to exercise the software artifact, not to model any particular person’s finances. The seeding routine walks the ninety-day window one day at a time. On the first of each month it inserts salary, on the fifth rent, around the twelfth utilities, and every seventeenth day a freelance payment. Ordinary daily expenses are drawn from a weighted category pool in which Groceries appears twice, with category-specific amount ranges in lek: 500 to 4,500 for Groceries, 800 to 3,500 for Restaurants, 200 to 1,500 for Transport, and up to 8,000 for Shopping. Merchant names come from small pools of recognisable Tirana businesses, including Spar and Big Market for groceries, Era and Mullixhiu for restaurants, Kastrati for fuel and Bolt for transport. The purpose is to make the demonstration screens resemble a plausible personal ledger rather than a collection of placeholder records.

III.2.3 Primary Data - Bank Statement Format Samples

For the importer, I collected statement-export samples from the online banking platforms of four major Albanian retail banks: BKT, Raiffeisen Bank Albania, Credins Bank, and Tirana Bank. The selection was purposive: four major banks were chosen so that the importer would be built against the statement formats most local users are likely to encounter. The samples informed the column auto-detection heuristics, the date formats supported, and the amount parsing. They were used only as schema references; no real transaction content was kept.

Three observations from the samples directly informed the importer design. The files use more than one delimiter, so the backend detects comma- versus semicolon-separated input rather than assuming one. Header language also varies: one sample may use English labels while another uses “Data”, “Shuma” and “Përshkrimi”. Finally, cell formats are heterogeneous: dates appear in several common representations, and amounts may use either European or US thousands/decimal separators, may be wrapped in accounting-style parentheses, and may retain currency text. Section IV.4.2 shows how these observations are handled by the pipeline.

III.2.4 Reproducibility of the Data

The demonstration data can be regenerated from the repository. The pseudo-random seed is fixed, so the same execution date produces the same generated records, and Appendix B gives the regeneration command. Backend package versions are pinned in requirements.txt, while frontend dependency resolution is recorded in package-lock.json. Date-dependent data windows and machine-dependent timing results remain documented sources of variation.

III.3 Tools and Technologies

Table 2 summarises the main technologies used in the implementation and the reason each was selected.

Table 2: Tools and technologies used in the implementation

Category	Tool / Technology	Reason for Choice
Backend language	Python 3.11	Mature web-development ecosystem, readable syntax, used widely in undergraduate education.
Web framework	Flask 3.0.3	Small framework surface suited to a REST API at this project scale; mature documentation and ecosystem (Grinberg, 2018).
ORM	SQLAlchemy 2 via Flask-SQLAlchemy 3.1.1	Mature Python ORM that maps relational data to application models while still permitting explicit SQL where needed.
DB migrations	Flask-Migrate 4.0.7 (Alembic)	Schema evolution under version control; reproducible across deployments.
Authentication	Flask-JWT-Extended 4.6.0	Standard JWT (Jones et al., 2015) implementation with access + refresh token pair support.
Password hashing	Flask-Bcrypt 1.0.1	Implements bcrypt (Provos & Mazières, 1999) with a deliberately costly work factor; bcrypt is retained as a project-level legacy/fallback choice consistent with current OWASP guidance (OWASP Foundation, n.d.).
Data validation	Custom validators module	Input shapes are small and stable; ~100 lines of require/parse/validate helpers

Category	Tool / Technology	Reason for Choice
		proved clearer than per-endpoint schema configuration.
Database	PostgreSQL 15	ACID-compliant relational database with CHECK constraints, ENUM types and support for the aggregation queries used by the reporting layer.
Frontend library	React 18.3.1	Component-based frontend library suited to the single-page application structure used in this project.
Build tool	Vite 5.4.2	Fast dev server with hot module replacement; production build via Rollup.
CSS framework	Tailwind CSS 3.4.10	Utility-first; produces small bundles via PurgeCSS-like JIT; permits design-system tokenisation.
Charting	Recharts 2.12.7	Composable React chart library built on D3; supports the chart types this thesis requires.
HTTP client	Axios 1.7.4	Interceptor support enables transparent JWT refresh.
Routing	React Router 6.26.1	Standard SPA client-side routing.
Testing	pytest 8.3.2 + pytest-flask 1.3.0	Widely used Python testing framework; pytest-flask provides integration with the Flask test client.
Deployment	Docker 24, Docker Compose 2	Reproducible multi-service deployment; one command brings up the full stack.
Production WSGI	Gunicorn 22.0	Production-grade WSGI server (Gunicorn Project, 2024).
Version control	Git + GitHub	Distributed version control and hosted repository used for source history, review and project distribution.

III.3.1 Justifying the Stack as a Whole

At the stack level, the main priorities were reproducibility, maintainability and the use of mature open-source components. None of the selected tools requires a proprietary licence, and each has established documentation and a substantial user community. The combination also avoids relying on experimental dependencies for core application functions. These choices are consistent with established discussions of maintainability, deployability and other software-quality attributes (Bass et al., 2021), (Fowler, 2002).

III.3.2 Alternatives Considered at the Tool Level

Two substitutions were considered at the tool level. Django could have provided a more opinionated application structure and built-in administration features, but Flask offered a smaller framework surface suited to the API-focused scope of this project. SQLite could have simplified deployment by removing the database container, whereas PostgreSQL provides the production data types, constraints and deployment model chosen for the application. SQLite is still used for the isolated in-memory test database because its zero-setup behaviour is well suited to deterministic test cases.

III.4 Evaluation Metrics

Chapter VI evaluates the system with software-engineering measures rather than inferential statistics, which is appropriate for assessing a design-science artifact against explicit implementation objectives (Hevner et al., 2004). The four metric categories are shown in Table 3.

Table 3: Evaluation metrics by category

Category	Metric	Operationalisation
Functional coverage	Feature-to-objective matrix	Every objective from Chapter I.4 must map to at least one implemented feature, demonstrated in Chapter VI.
Correctness	Automated test pass rate	pytest suite executed against an in-memory SQLite database; target: 100% of cases pass.
Security	Cross-user isolation test	Dedicated test case attempts to access user A's data while authenticated as user B; must return 0 results / 404.

Category	Metric	Operationalisation
Performance	Endpoint response latency	Three cold-start requests are discarded; the median of the seven remaining warm requests is reported per endpoint.

Server-side latency is recorded through Flask request hooks. A before-request hook records the start time and an after-request hook calculates elapsed time for the completed request. This keeps the measurement independent of browser rendering, but it excludes network delay and client-side processing; that limitation is discussed in Section VI.4.3.

III.4.1 Justifying the Metric Choices

Functional coverage is assessed by mapping each objective to implemented features. Automated tests provide repeatable evidence for selected behaviours, with emphasis on authentication, transaction integrity, CSV-import behaviour and cross-user isolation rather than exhaustive correctness (Ammann & Offutt, 2016). Cross-user isolation is reported separately because ownership boundaries are critical in a multi-user data model. These measures correspond broadly to functional suitability, performance efficiency and security characteristics in ISO/IEC 25010 (ISO/IEC, 2023). Nielsen’s one-hundred-millisecond threshold is used only as an interpretive response-time reference (Nielsen, 1993).

The evaluation deliberately omits several areas. No load test was performed beyond the demonstration dataset, so the results do not establish long-term scalability or behaviour under high concurrency. No formal usability study with recruited participants was conducted either. The latency figures are therefore evidence about server-side performance, not a complete measure of user-perceived usability (ISO/IEC, 2023).

Chapter IV: System Design and Architecture

IV.1 Architectural Overview

Vault uses a three-tier architecture consistent with established software-architecture practice (Eckerson, 1995), (Bass et al., 2021), (Fowler, 2002). The presentation tier is a React single-page application; in the supplied Docker Compose environment it is served by the Vite development server, while a production deployment can serve the built static assets through a conventional web server. The application tier is Flask served by Gunicorn (Gunicorn Project, 2024) in the supplied Compose stack, exposing a REST API under `/api`. The data tier is PostgreSQL 15, reached through SQLAlchemy. Docker Compose runs the three services separately, providing a reproducible deployment structure. The separation of presentation, application logic and data management supports maintainability, deployability and controlled performance trade-offs (Bass et al., 2021), (Fowler, 2002), while the persistence choices follow established discussion of reliable data systems (Kleppmann, 2017).

Figure 2 provides an overview of the three-tier structure and shows how the React frontend, Flask application layer and PostgreSQL data layer interact within the deployed system.

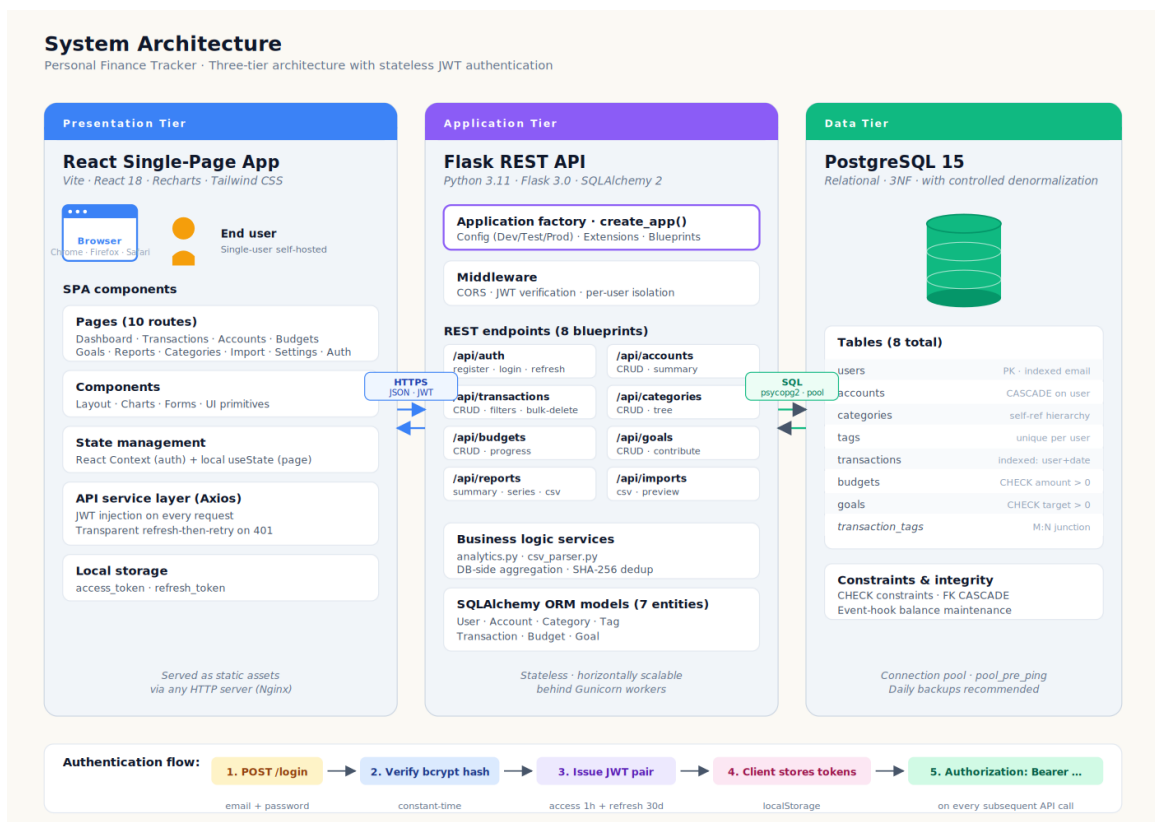


Figure 2: System architecture (three-tier)

The three tiers have deliberately separated responsibilities. The frontend handles presentation, interaction state, token-refresh behaviour and CSV column-guessing assistance. Business-critical authorisation, persistence and financial rules are enforced by the backend, which derives user identity from the verified token rather than trusting a client-supplied user identifier. The database enforces referential integrity through foreign keys, CHECK constraints and ENUM types, while application-level business rules remain in code where they can be tested and changed without a schema migration.

IV.2 Entity-Relationship Model

The data model contains seven entity tables plus one junction table. It is predominantly normalised, with `current_balance` retained as a deliberate denormalisation discussed in Section IV.5; the relational design follows established normalisation principles (Codd, 1970), (Elmasri & Navathe, 2016). Table **Error! Reference source not found.** reports the seeded row counts used during evaluation. Figure 3 summarises the main entities, ownership relationships and links between transactions, accounts, categories, budgets, goals and tags in the implemented data model.

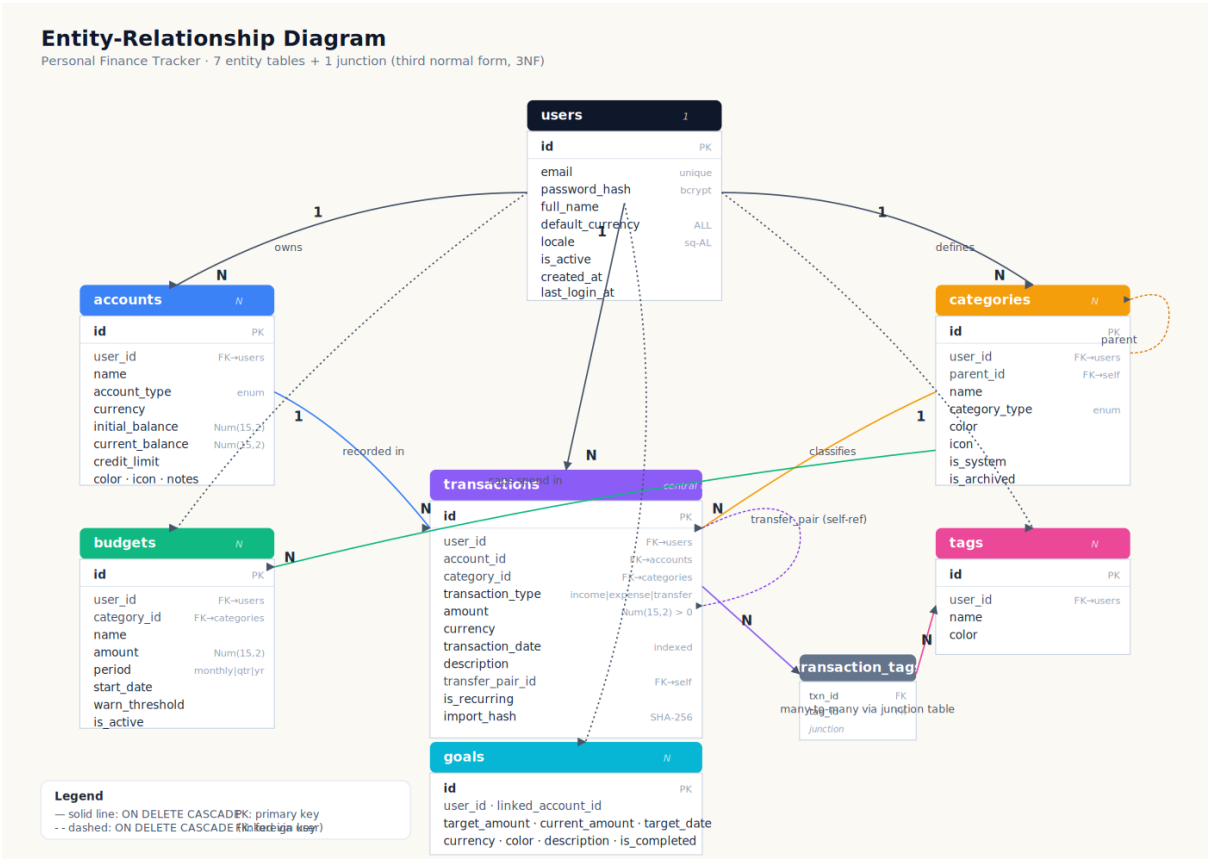


Figure 3: Entity-relationship diagram

Three design choices are central to the data model. Every user-owned table has a non-null `user_id` with `ON DELETE CASCADE`, so deleting a user removes the records they own; selected optional relationships, such as category or linked-account references, use `SET NULL` when the owning record should remain. Monetary amounts are stored as positive `Decimal(15,2)` values, with direction represented by the `transaction_type` enum. Transfers are stored as a pair of transactions connected by a `transfer_pair_id` self-reference, allowing each side of the transfer to remain visible in the relevant account history without introducing a separate transfer entity.

IV.3 Use Case Overview

The primary actor is the individual user. No admin role exists, which follows the individual/self-hosted personal-use scope in Chapter I.5. Multi-user data isolation is nevertheless implemented and tested; shared household features are out of scope.

Figure 4 presents the main interactions available to the individual user, from authentication and transaction management to budgeting, reporting, import and application settings.

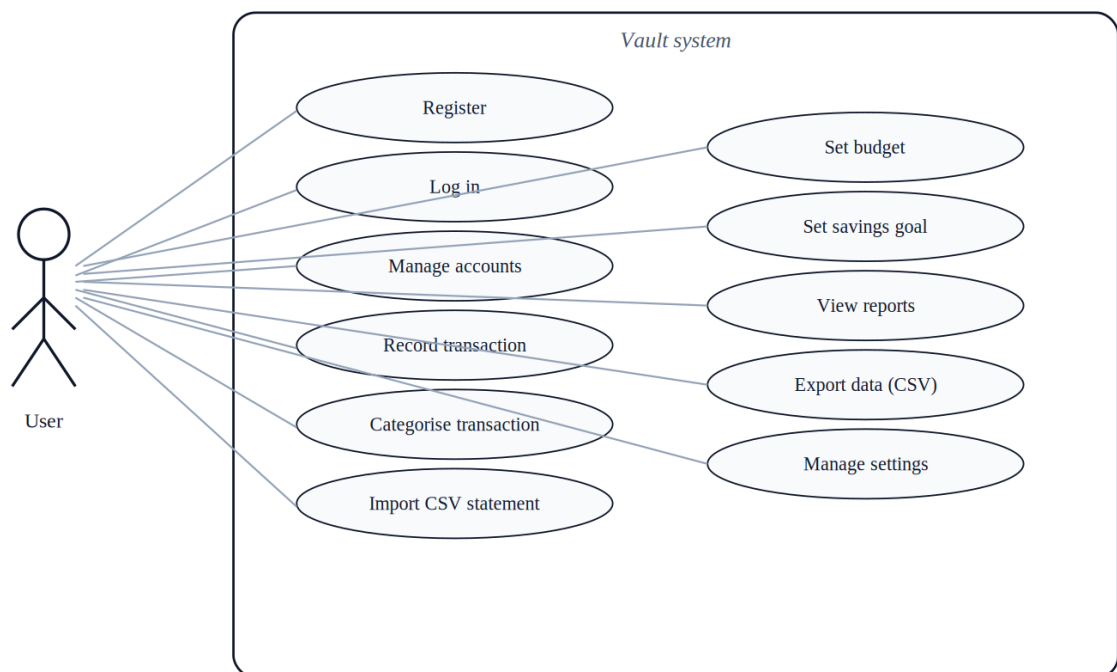


Figure 4: Use case overview

The system is organised around five main use cases:

Onboarding and authentication. The user registers, logs in, and receives a JWT access and refresh token pair.

Accounts and transactions. The user creates accounts in their preferred currencies and records transactions, by hand or through CSV import.

Budgeting. The user caps spending per category over a monthly, quarterly or yearly period and watches progress against the cap.

Goals. The user names a savings goal, sets a target amount and date, and contributes toward it over time.

Reporting and export. The user reads aggregated reports over a chosen time window and can export the underlying transactions as CSV.

IV.4 Sequence Diagrams

Login and CSV import are shown at sequence level because both workflows cross the frontend, API and persistence boundaries and involve protocol or validation steps that are not visible in the use-case diagram.

IV.4.1 Authentication Flow

Figure 5 illustrates the authentication sequence from credential submission to token issuance and the use of the resulting access token in subsequent protected API requests.

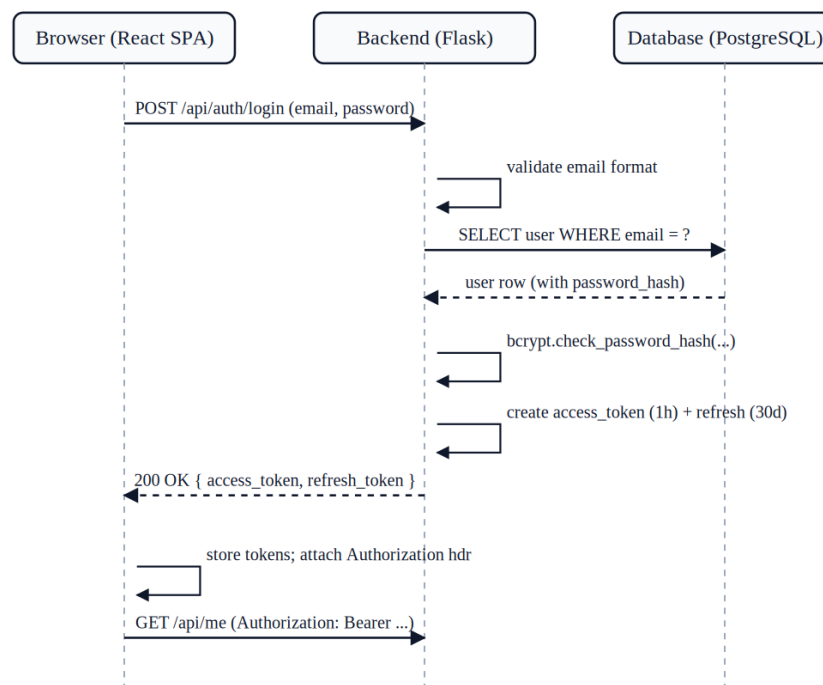


Figure 5: Sequence diagram: user login and JWT issuance

After login, every API call carries the access token in the Authorization header, and the backend JWT middleware checks the signature and expiry before any handler runs. The access token

lives one hour. When it expires, an Axios interceptor exchanges the refresh token (thirty-day lifetime) for a new access token and replays the failed request. This design keeps access credentials short-lived while avoiding a server-side session store. The JWT format and implementation considerations are documented in RFC 7519 and RFC 8725 (Jones et al., 2015), (Jones et al., 2020).

IV.4.2 CSV Import Pipeline

Figure 6 summarises the two-stage CSV import workflow, showing the preview and column-mapping step followed by parsing, duplicate checking and persistence of accepted rows.

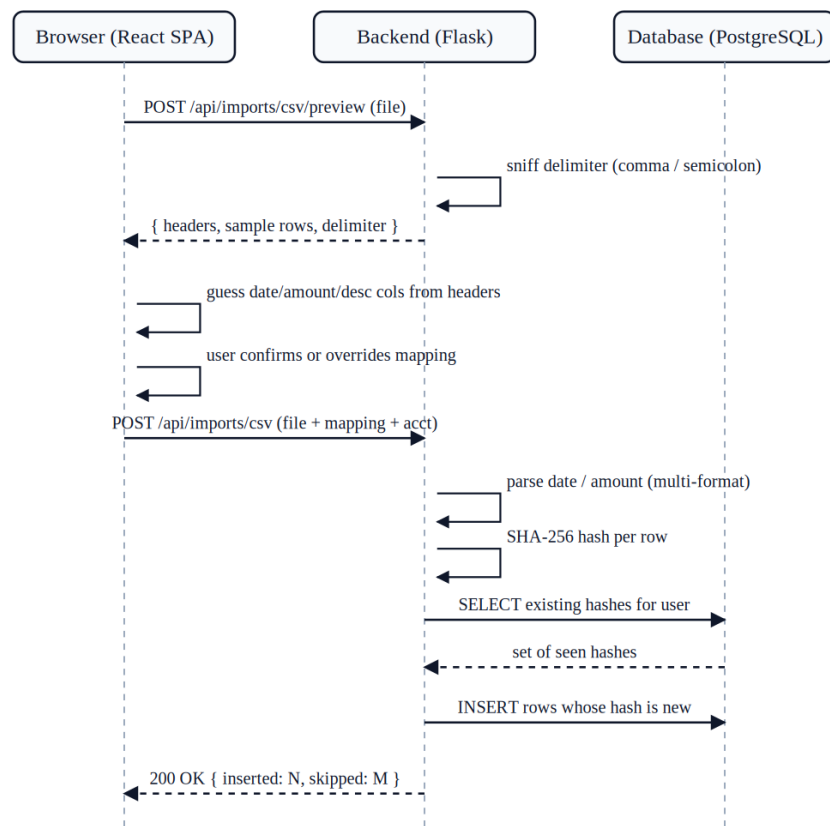


Figure 6: Sequence diagram: CSV import pipeline

Import happens in two phases. In the preview phase the backend detects the delimiter because the sampled statements use both commas and semicolons, then returns the headers and initial rows. The React import page suggests column mappings by matching headers against conservative expressions: date|datum for dates, amount|sum|value|debit|credit for amounts, and desc|name|memo|narration|reference for descriptions. Albanian headers such as “Data”, “Shuma” and “Përshkrimi” do not match these patterns and therefore fall through to manual selection. After the user confirms the mapping, the commit phase parses the complete file,

supports DD/MM/YYYY, DD.MM.YYYY, DD-MM-YYYY, YYYY-MM-DD and written month forms, accepts US and European decimal conventions plus accounting-style negatives, and computes a SHA-256 row hash. A hash already present for the same user is skipped, making re-import of overlapping statement data idempotent. Chapter VI evaluates these behaviours through five dedicated importer tests.

IV.5 Technology Stack and Design Decisions

Five design decisions required explicit trade-offs during development. Table 4 summarises the problem, alternatives and chosen approach for each decision.

Table 4: Design decisions, alternatives considered, and chosen approaches

#	Problem	Alternatives considered	Chosen approach and reason
1	Where to maintain account balances	(a) Recompute on every read by summing all transactions; (b) Maintain a denormalised <code>current_balance</code> column updated on every transaction insert/delete.	(b). Recomputing every account balance from transaction history on each dashboard read would add repeated aggregate work. Maintaining <code>current_balance</code> shifts that work to transaction writes through SQLAlchemy hooks and provides direct balance reads for the interactive interface. The trade-off is controlled denormalisation whose consistency must be tested.
2	Sign convention for transaction amounts	(a) Positive amounts with sign implied by <code>transaction_type</code> ; (b) Signed amounts on a single column.	(a). Both representations are viable. This project stores a positive monetary magnitude and represents direction through <code>transaction_type</code> , keeping amount validation separate from income/expense semantics and applying direction explicitly in reporting logic.
3	Currency conversion strategy	(a) Convert at write time to one base currency; (b) Convert at read time using an exchange-rate source; (c) Preserve	(c). The implementation preserves the recorded currency rather than embedding an unverified exchange-rate policy. Per-currency totals remain available and net-worth reporting uses the user's default currency. Other

#	Problem	Alternatives considered	Chosen approach and reason
		native currency and avoid automatic FX conversion.	cross-currency analytical views require a common reporting-currency interpretation until an explicit FX layer is added.
4	CSV importer architecture	(a) Per-bank parser classes; (b) Generic importer with user-supplied column mapping and heuristic auto-detection.	(b). Statement formats can change between banks and over time, and even accounts at one institution may export different columns. A generic mapping step allows the user to confirm the meaning of the file's columns while the parser remains independent of a particular bank format.
5	Authentication mechanism	(a) Server-side sessions backed by Redis or similar; (b) Stateless JWT with access + refresh token pair.	(b). For an individual self-hosted deployment, JWT avoids a separate shared session store and keeps authentication state in signed access and refresh tokens. This simplifies the application tier, but production hardening still requires secure secret management, HTTPS and an explicit token-revocation strategy if server-side logout invalidation is required.

Chapter V: Implementation

Implementation follows the architecture defined in Chapter IV. The discussion covers the development environment, backend and database structure, API conventions, frontend organisation and automated tests. Only short code excerpts are included here; the complete source is referenced in Appendix A.

V.1 Development Environment Setup

Development took place on Windows 11, partly natively and partly under WSL2. The documented environment is reproduced by cloning the repository and starting the three-service Docker Compose stack: PostgreSQL, the Gunicorn-served Flask backend, and the Vite development server for the frontend.

```
# Reproducing the development environment
git clone https://github.com/Krisjor/Vault-Finance-Tracker finance-tracker
cd finance-tracker
docker-compose up -d --build
docker-compose exec backend flask --app run db upgrade
docker-compose exec backend flask --app run seed-demo
```

After these commands, the backend is available at <http://localhost:5000>, the frontend at <http://localhost:5173>, and PostgreSQL at localhost:5432. The seed-demo command creates the demonstration user (demo@example.com, password password123) with the ninety-day dataset from Section III.2.2. Startup time varies with machine speed, network state and whether container images have already been built.

V.2 Backend Code Organisation

The backend is organised into approximately twenty-eight application modules under backend/app/, grouped by domain: models/ contains entity models, api/ Flask Blueprints, services/ cross-cutting logic such as analytics and CSV parsing, seeds/ data seeding, and utils/ shared validation and authentication helpers. Flask extensions - SQLAlchemy, Flask-Migrate, Flask-JWT-Extended, Flask-Bcrypt and Flask-CORS - are instantiated centrally in extensions.py so the application factory and model modules can share them without circular initialization dependencies.

Figure 7 shows how the application factory brings together configuration, Flask extensions, API Blueprints, error handling and runtime entry points to produce the configured backend application.

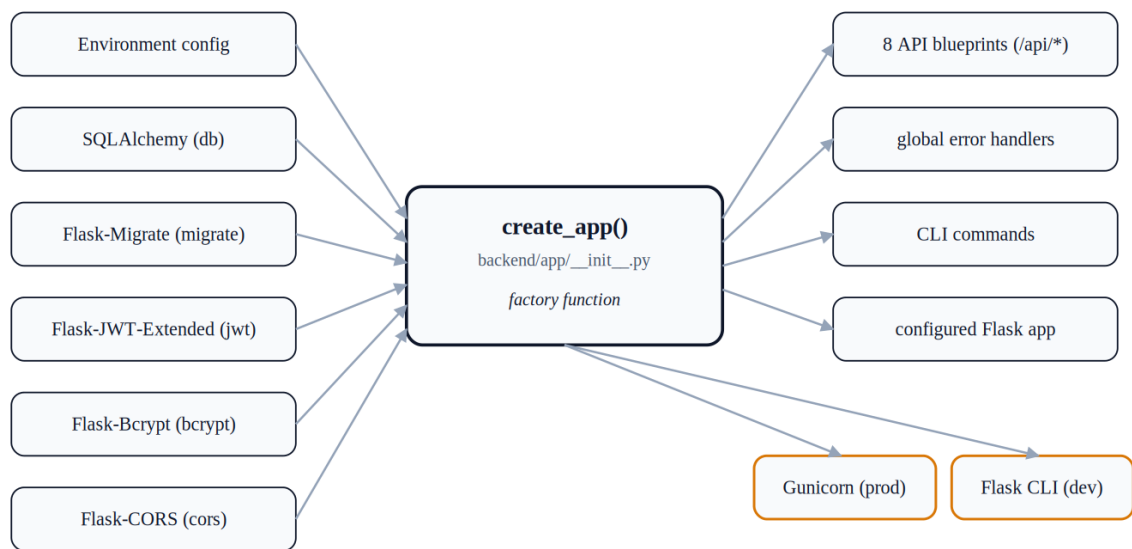


Figure 7: Application-factory pattern in the Flask backend

Code 5.1 shows the User model in abbreviated form. User ownership is central to the schema: owned resources reference `user_id`, and the relationships are configured so that deleting a user removes dependent owned data while optional non-ownership links use more conservative deletion behaviour.

```

# Code 5.1 - backend/app/models/user.py (abbreviated)
class User(db.Model):
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(255), unique=True,
                      nullable=False, index=True)
    password_hash = db.Column(db.String(255), nullable=False)
    default_currency = db.Column(db.String(3),
                                  default="ALL", nullable=False)
    accounts = db.relationship("Account", backref="user",
                               cascade="all, delete-orphan")

    def set_password(self, plaintext: str) -> None:
        self.password_hash = bcrypt.generate_password_hash(
            plaintext).decode("utf-8")

    def check_password(self, plaintext: str) -> bool:
        return bcrypt.check_password_hash(self.password_hash, plaintext)

```

V.3 Database Implementation

The schema is declared through SQLAlchemy models and versioned with Alembic migrations. The initial migration creates the seven entity tables, the junction table, and five PostgreSQL

ENUM types backing string-valued Python Enum classes in the models. Table 5 shows each table with its row count on the seeded data.

Table 5: Database tables and their cardinality on the demonstration dataset

Table	Purpose	Demo rows
users	User accounts; one per registered user	1
accounts	Financial accounts owned by users (checking, savings, etc.)	4
categories	Income and expense categories (system + user-defined)	16
tags	User-defined transaction tags	0
transactions	Income, expense, and transfer records	~150
transaction_tags	Junction table for the transactions ↔ tags many-to-many relation	0
budgets	Per-category spending caps with period (monthly/quarterly/yearly)	5
goals	Named savings goals with target amount and optional target date	3

Three constraint patterns recur across the schema: non-null user ownership with ON DELETE CASCADE for owned records, NUMERIC(15,2) for money so values are not stored as binary floating point, and PostgreSQL ENUM columns for controlled values. The relational design follows Codd (Codd, 1970), with practical 3NF and update-anomaly guidance from Elmasri and Navathe (Elmasri & Navathe, 2016). One implementation issue involved ENUM serialisation: the model enums subclass str and enum.Enum, so TransactionType.INCOME has the value “income”, while SQLAlchemy can otherwise serialise the member name. The migration initially exposed this mismatch. The fix was to use values_callable on each Enum column so SQLAlchemy serialises the enum value rather than the member name.

V.4 API Layer

The application exposes forty-three resource/domain routes across eight Flask Blueprints, plus a global /api/health endpoint for liveness checks. The Blueprints provide create, read, update and delete operations where appropriate to each resource, together with domain-specific actions

such as goal contributions, transaction bulk deletion, report export and CSV import. A consistent JSON error convention and per-user ownership filtering are applied across protected resources. Table 6 gives the route breakdown.

Table 6: API endpoint surface by resource

Blueprint	URL prefix	Endpoints	Notes
auth	/api/auth	6	register, login, refresh, logout, me (GET), me (PATCH)
accounts	/api/accounts	6	list, get, create, update, delete, summary
categories	/api/categories	5	list, tree (hierarchical view), create, update, delete
transactions	/api/transactions	6	list (paginated), get, create, update, delete, bulk-delete
budgets	/api/budgets	5	list, get, create, update, delete
goals	/api/goals	5	list, create, update, delete, contribute
reports	/api/reports	8	summary, spending-by-category, income-by-category, monthly-series, net-worth, top-merchants, daily-spending, export.csv
imports	/api/imports	2	csv/preview, csv (full commit)

Code 5.2 shows the transaction-creation endpoint in abbreviated form. The `login_required` decorator validates the JWT and supplies the resolved user to the handler, so ownership is derived from the token rather than a client-supplied `user_id`. Request values are checked by a small `validators` module containing helpers such as `require`, `parse_decimal`, `parse_date` and `validate_currency`; validation failures return HTTP 400 with the relevant field identified. Referenced resources are queried together with the authenticated user ID. Marshmallow was considered for schema validation, but the request structures in this project are small and stable, so a compact shared validator module was kept instead.

```
# Code 5.2 - backend/app/api/transactions.py (abbreviated)
@transactions_bp.route("", methods=["POST"])
@login_required
def create_transaction(user):
    data = request.get_json(silent=True) or {}
```

```

try:
    require(data, "account_id", "transaction_type",
            "amount", "transaction_date")
    ttype = TransactionType(data["transaction_type"])
    account = _ensure_user_owns_account(user, data["account_id"])
    amount = parse_decimal(data["amount"], "amount")
    # ... category type check and transfer pairing elided ...
    txn = Transaction(user_id=user.id, account_id=account.id,
                     transaction_type=ttype, amount=amount, ...)
    db.session.add(txn)
    db.session.commit()
    return jsonify(txn.to_dict()), 201
except ValidationError as e:
    return jsonify(error="validation", field=e.field,
                  message=e.message), 400

```

Across protected API routes, the same ownership rules are applied: authentication is required, the user identity is obtained from the verified token, and queries over owned resources include that user ID. The dedicated isolation test in Section V.6 checks these rules through both list and direct-retrieval requests.

V.5 Frontend Implementation

The frontend is a React 18 single-page application built with Vite. It follows the component patterns Banks and Porcello describe (Banks & Porcello, 2020): eleven page components, one per route, a shared layout (Sidebar plus content area), a handful of reusable primitives (Modal, StatCard, Toast, Icon), an authentication context, and an Axios client that handles tokens.

Figure 8 visualises the frontend component hierarchy, from the application and authentication context down to routed pages and reusable interface components.

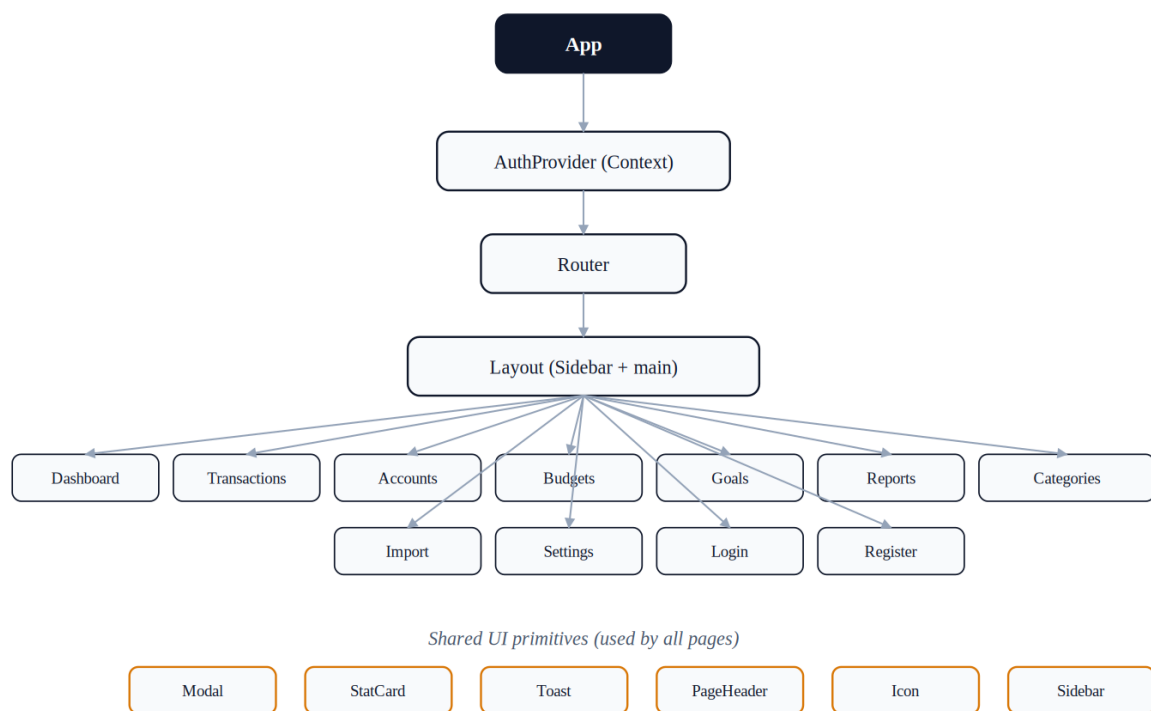


Figure 8: React component hierarchy

No separate global client-state library such as Redux or Zustand is used. React Context stores the small amount of state that must be shared across routes, including the authenticated user and tokens, while filters, modal state and form values remain local to individual pages. Pages request authoritative data from the API when needed. This keeps client state simple, at the cost of additional API requests whose server-side latency is measured in Chapter VI.

Code 5.3 shows the Axios interceptor used for token refresh. A 401 response triggers one refresh attempt and then replays the original request; if refresh fails, the error is returned normally. The complete implementation also queues concurrent 401 responses behind a single in-flight refresh so that multiple failed requests do not create multiple refresh calls.

```

// Code 5.3 - frontend/src/services/api.js (abbreviated)
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry
        && tokenStore.getRefresh()) {
      original._retry = true;
      const refresh = tokenStore.getRefresh();
      const res = await axios.post(
        `${API_BASE}/auth/refresh`, {}

```

```

    { headers: { Authorization: `Bearer ${refresh}` } });
    tokenStore.set({ access_token: res.data.access_token });
    return api(original); // replay original request
  }
  return Promise.reject(error);
}
);

```

V.5.1 Dashboard Screen

Figure 9 presents the main dashboard view, where summary indicators, recent financial trends, category-level spending and account information are combined in a single overview.

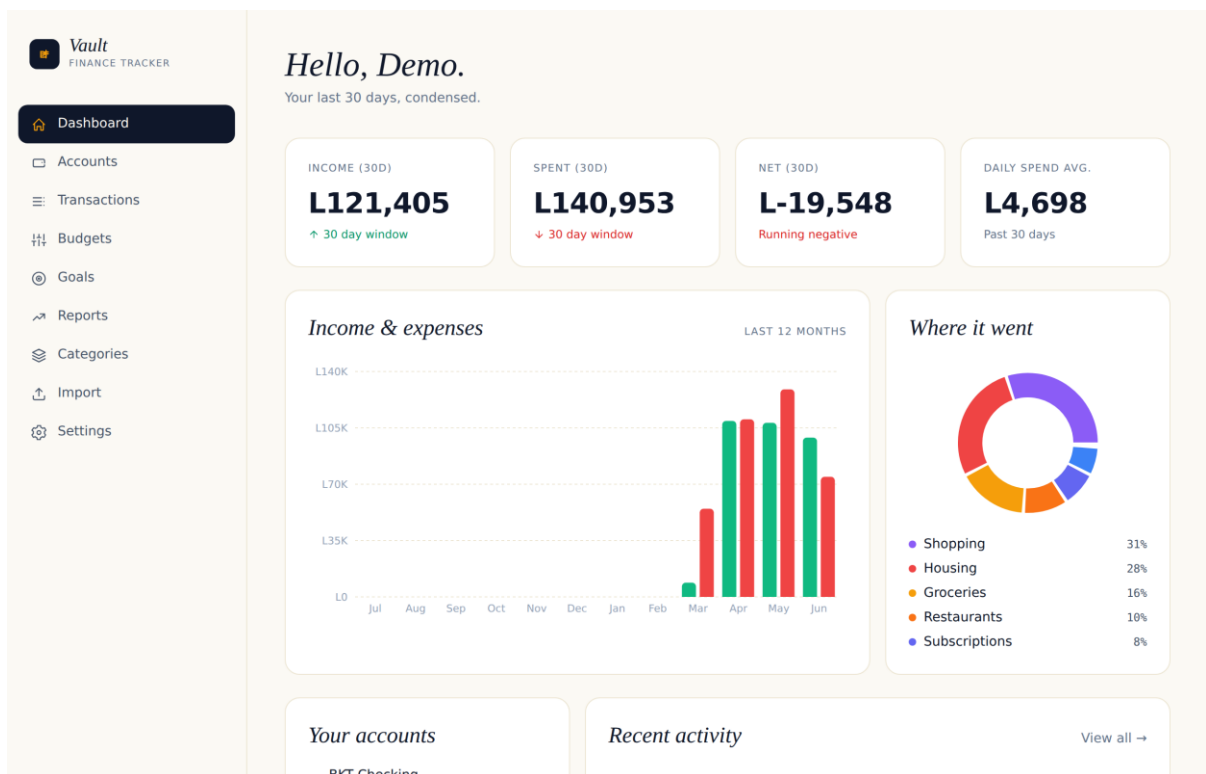


Figure 9: Dashboard screen

The dashboard performs four requests in parallel: `/api/reports/summary` for the main analytical payload, `/api/transactions` for recent activity, `/api/accounts` for account information, and `/api/categories` for category metadata. The summary endpoint itself returns period totals, spending by category, a twelve-month series and average daily spending. Keeping these concerns in separate API resources preserves the existing API structure while `Promise.all` allows the frontend to load them concurrently.

V.5.2 Transactions Screen

Figure 10 shows the transaction-management view, including date-based grouping and the filters used to narrow the displayed records by account, category, type or search text.

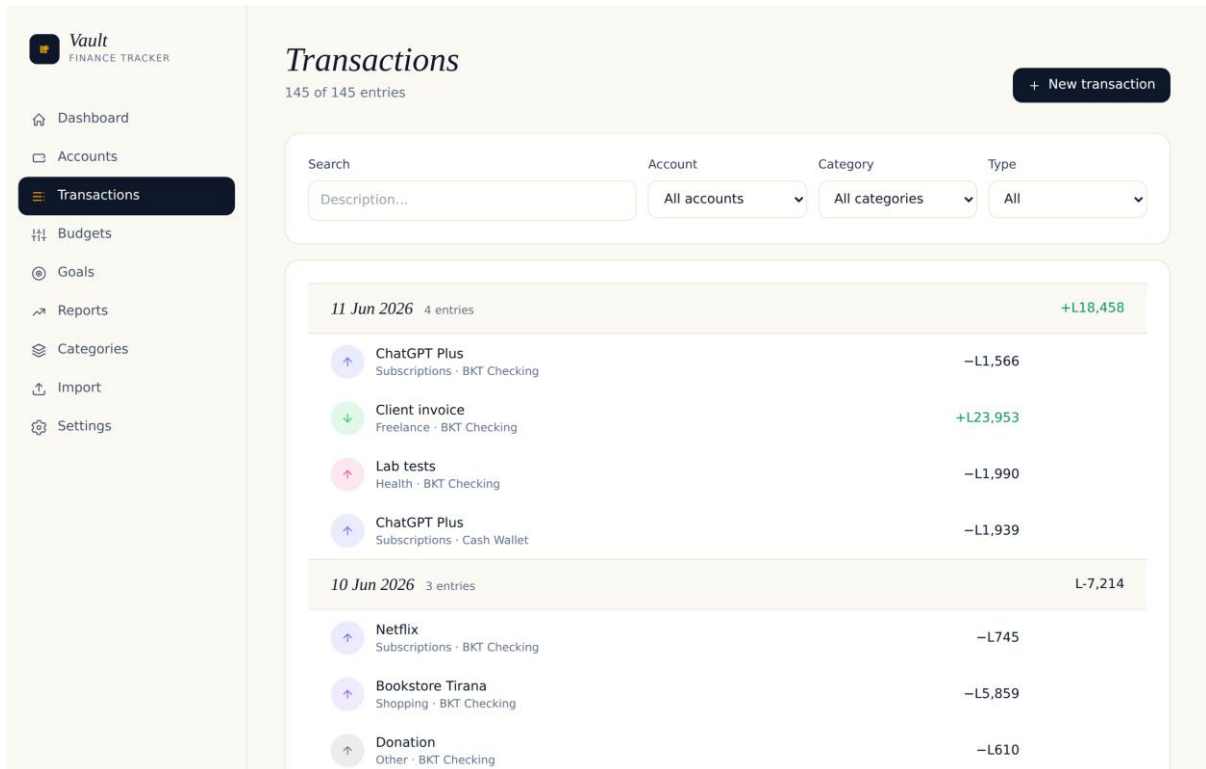


Figure 10: Transactions screen with date grouping

Transactions are grouped by date with a per-day net total in each group header, and each row exposes edit and delete controls. Filtering by search text, account, category and type is performed server-side through query parameters on GET /api/transactions. This avoids loading the full history into the browser, although performance over multi-year or high-volume datasets was not measured in this thesis.

V.5.3 Budgets and Goals Screens

Figure 11 illustrates the budget-monitoring interface, where each category is shown with its spending limit, current expenditure and a visual progress indicator.

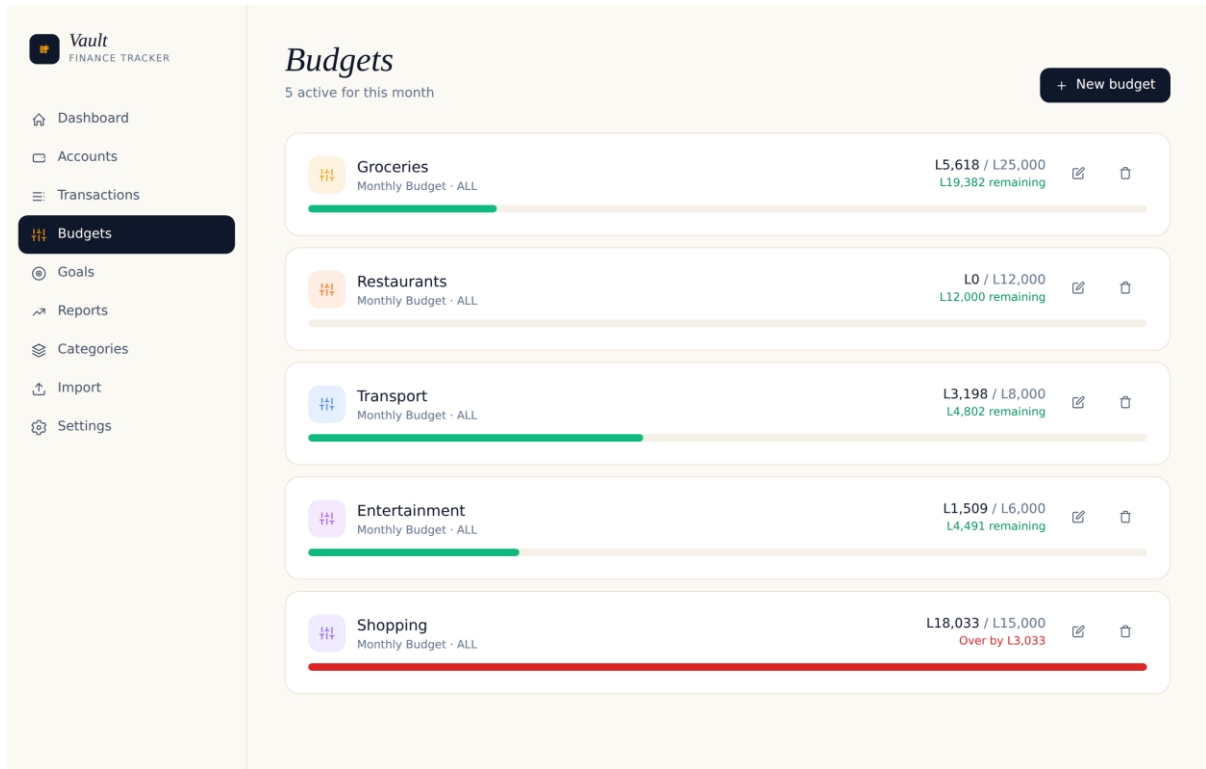


Figure 11: Budgets screen with progress indicators

Each budget is displayed with its spent amount, limit and a progress bar that changes from green to amber at the warning threshold (eighty percent by default) and to red when the limit is exceeded. In Figure **Error! Reference source not found.**, the Shopping budget exceeds its limit by 3,033 lek, so the over-budget state is visible both numerically and through the progress indicator. Savings goals use a similar current-versus-target presentation and include a contribution action.

V.6 Testing

Tests use pytest with the pytest-flask extension: sixteen cases in three files. test_auth.py contains seven cases, test_transactions.py four, and test_imports.py five dedicated importer cases. The importer tests cover comma and semicolon previews, four date representations, US and European amount formats including an accounting-style negative, and SHA-256 duplicate detection/skip behaviour. Each test receives a fresh in-memory SQLite database from the confest fixture, so state does not leak between cases; this follows established test-isolation practice (Meszaros, 2007). Passing tests provide repeatable evidence for the exercised behaviours but do not establish correctness for untested behaviours (Ammann & Offutt, 2016).

```
# Code 5.4 - backend/tests/test_transactions.py (abbreviated)
def test_user_cannot_see_other_users_transactions(client, db):
    h1 = _auth_header(client, db, email="user4@example.test")
    h2 = _auth_header(client, db, email="user5@example.test")
    a1 = _create_account(client, h1)
    txn = client.post("/api/transactions", json={...},
                     headers=h1).get_json()
    listing = client.get("/api/transactions", headers=h2).get_json()
    assert listing["total"] == 0
    res = client.get(f"/api/transactions/{txn['id']}", headers=h2)
    assert res.status_code == 404
```

Code 5.4 exercises a critical ownership boundary. Both users are registered inside the test through local helper calls, after which user B is checked against both the transaction list and a direct fetch of user A’s transaction. The expected results are zero visible transactions and HTTP 404 for the direct lookup.

Chapter VI: Evaluation and Results

The completed application is evaluated against the six objectives defined in Section I.4. The evaluation covers functional completion, automated test results and measured server-side response time, followed by an assessment of the limits of that evidence.

VI.1 Experimental Setup

VI.1.1 Hardware and Software Environment

Everything was measured on my development machine: an Intel Core i5-11400 (six cores, twelve threads, 2.6 GHz base, 4.4 GHz boost), 16 GB of DDR4, a 512 GB NVMe SSD, Windows 11 Pro 24H2 with Docker Desktop 4.34 on the WSL2 backend. PostgreSQL 15.4 ran in its own container; the backend ran Python 3.11.9 under the Flask development server (not Gunicorn); the frontend ran the Vite 5.4.2 dev server. The browser for end-to-end checks was Microsoft Edge 130. The machine was otherwise idle during measurement.

VI.1.2 Test Database

The automated tests use an in-memory SQLite database created fresh per test from the SQLAlchemy metadata via `db.create_all()`, without executing migrations. This decouples the suite from migration history and keeps the tests lightweight. The project Docker image pins Python 3.11 and pytest 8.3.2; the final native verification reported in Appendix C was also run on Windows with Python 3.14.7 and pytest 9.1.1, where all sixteen cases passed in 6.79 seconds. Performance measurements, in contrast, use the Dockerised PostgreSQL seeded environment: each endpoint is hit ten times, the first three responses are discarded as cold-start noise, and the median of the remaining seven is reported.

VI.1.3 Baselines

The measurements are interpreted against two reference points. Nielsen's one-hundred-millisecond threshold (Nielsen, 1993) is used as a reference for ordinary interactive response time. Authentication is considered separately because bcrypt password verification is intentionally expensive: the project uses work factor 12, while OWASP advises selecting the highest practical work factor and gives 10 as a minimum for bcrypt where it is used (OWASP Foundation, n.d.). Login is therefore expected to take longer than ordinary data-retrieval requests.

VI.2 Results

VI.2.1 Functional Coverage

Table 7 maps the six objectives to what was built. Every objective has at least one implemented feature behind it.

Table 7: Feature-to-objective traceability matrix

#	Specific objective	Implemented features	Status
1	Predominantly normalised 7-entity relational model with controlled balance denormalisation	Seven entity tables + transaction_tags junction; user-owned entities cascade-delete with the user; selected optional relationships use SET NULL; current_balance is deliberately denormalised and maintained through SQLAlchemy hooks.	Complete
2	REST API with JWT, bcrypt and per-user isolation	Eight API Blueprints with 43 resource/domain routes plus /api/health; access + refresh tokens via Flask-JWT-Extended; bcrypt work factor 12; protected owned-data access derives identity from the token and applies user filtering.	Complete
3	Analytical reporting with sub-100 ms ordinary interactive target at demonstration scale	Seven analytical report endpoints plus CSV export; principal SUM/GROUP BY work is database-side; ordinary interactive endpoints measured below the 100 ms reference in Section VI.2.3.	Complete
4	React SPA with multi-currency and editorial style	Eleven page components; Tailwind CSS design system; five-currency support (ALL, EUR, USD, GBP, CHF); Instrument Serif italic headings.	Complete
5	Generic CSV importer designed against major Albanian-bank statement samples	Two-phase preview/commit pipeline; manual/heuristic column mapping; multi-format date parsing; multi-locale amount parsing; SHA-256 per-user deduplication; no per-bank parser classes.	Complete

#	Specific objective	Implemented features	Status
6	Automated test suite covering selected critical behaviours	Sixteen pytest cases covering authentication, transaction integrity, CSV importer behaviour and cross-user isolation; all cases pass in the final native verification reported in Appendix C.	Complete

VI.2.2 Correctness via Automated Testing

All sixteen cases pass against the final code. Table 8 lists them; Figure 12 shows the same result by test file.

Table 8: Automated test results

File	Test case	Outcome
test_auth.py	test_register_creates_user_and_returns_tokens	PASS
test_auth.py	test_register_rejects_short_password	PASS
test_auth.py	test_register_rejects_duplicate_email	PASS
test_auth.py	test_login_succeeds_with_correct_credentials	PASS
test_auth.py	test_login_rejects_wrong_password	PASS
test_auth.py	test_me_requires_auth	PASS
test_auth.py	test_me_returns_user_when_authenticated	PASS
test_transactions.py	test_create_expense_decreases_balance	PASS
test_transactions.py	test_create_income_increases_balance	PASS
test_transactions.py	test_delete_transaction_restores_balance	PASS
test_transactions.py	test_user_cannot_see_other_users_transactions	PASS
test_imports.py	test_csv_preview_comma_delimited	PASS
test_imports.py	test_csv_preview_semicolon_delimited	PASS
test_imports.py	test_csv_date_parsing_formats	PASS
test_imports.py	test_csv_amount_parsing_locales	PASS
test_imports.py	test_csv_import_sha256_deduplication	PASS
TOTAL	16 cases, 0 failed	100%

Figure 12 provides a compact visual summary of the sixteen passing test cases, grouped across the authentication, transaction and CSV-import test files.

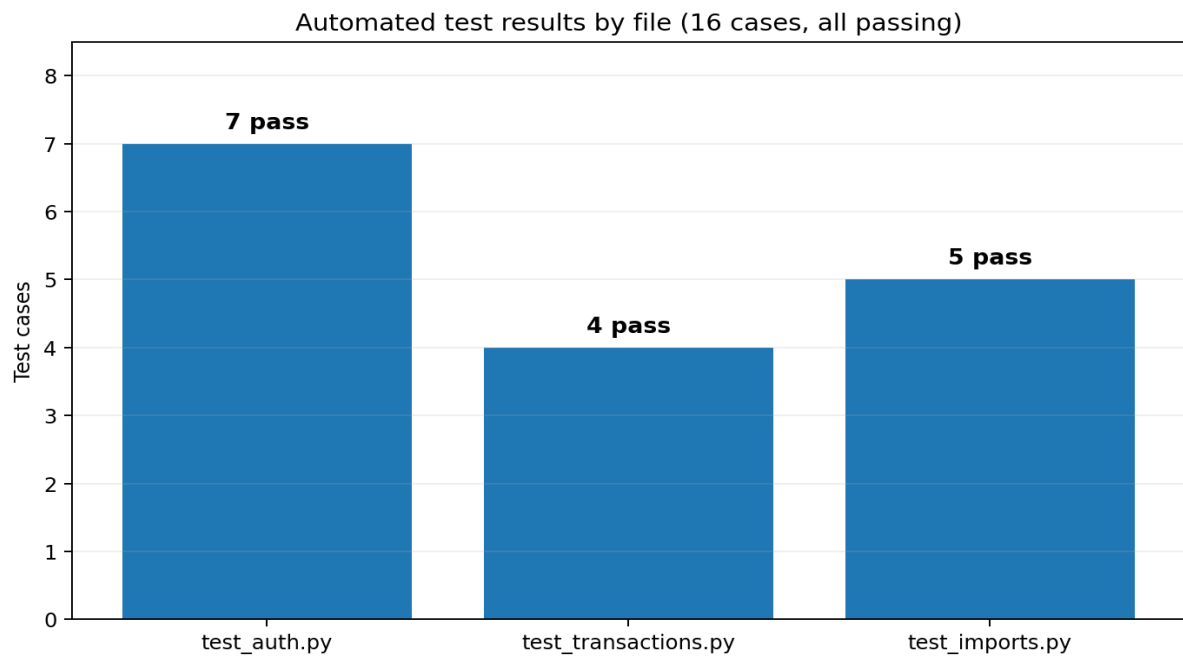


Figure 12: Automated test results by file

The passing suite establishes repeatable evidence for selected behaviours: passwords are hashed and verified through bcrypt; transaction creation and deletion maintain account balances in the exercised cases; cross-user transaction listing and direct retrieval are blocked; and the importer handles the tested delimiter, date, amount and duplicate scenarios. It does not establish exhaustive correctness, branch coverage, penetration resistance or security against threats that the suite does not exercise (Ammann & Offutt, 2016).

VI.2.3 Performance Characterisation

Table 9 gives the measured median server-side response times under the demonstration load, using seven warm requests per endpoint after three discarded requests.

Table 9: Endpoint response-time characterisation (median of 7 warm requests)

Endpoint	Method	Median latency	Notes
/api/auth/login	POST	≈280 ms	bcrypt-bound; expected (and desirable)
/api/accounts	GET	<10 ms	Indexed lookup
/api/accounts/summary	GET	<10 ms	Groups active net-worth account

Endpoint	Method	Median latency	Notes
			balances by currency in memory
/api/transactions?page=1&page_size=50	GET	20-40 ms	Indexed range scan on date
/api/transactions	POST	20-35 ms	Insert + cascade hooks for balance maintenance
/api/reports/summary	GET	40-80 ms	Four analytical results returned in one dashboard-summary call
/api/reports/spending-by-category	GET	15-30 ms	Database GROUP BY over category and scoped transaction rows
/api/reports/monthly-series	GET	30-50 ms	12-month database GROUP BY by extracted year/month
/api/reports/net-worth	GET	40-70 ms	Default-currency monthly database totals with cumulative calculation in application logic
/api/imports/csv (90-row file)	POST	150-250 ms	Includes SHA-256 hashing per row

The measured results show that the ordinary interactive API endpoints in the demonstration scenario fall below the hundred-millisecond reference, while login and CSV import are slower by design and workload. This is consistent with the operation of indexed queries and database-side aggregation in PostgreSQL (PostgreSQL Global Development Group, 2024). Login is the main authentication outlier at roughly 280 ms because bcrypt verification is intentionally computationally expensive (Provos & Mazières, 1999), (OWASP Foundation, n.d.). The CSV

import is slower because its work scales with the number of uploaded rows. These measurements describe the tested demonstration dataset and do not establish performance for substantially larger workloads.

VI.3 Discussion

VI.3.1 Interpretation of the Results

The automated results provide evidence that the selected authentication, transaction, importer and ownership behaviours operate as intended in the tested cases. In particular, the isolation case checks a user-ownership boundary, and the importer tests cover delimiter detection, four date formats, US and European amount notation, accounting-style negatives and duplicate skipping. This is targeted evidence rather than exhaustive correctness (Ammann & Offutt, 2016). The latency measurements separately show that ordinary interactive endpoints remain within the chosen response-time reference at demonstration scale, with the main analytical aggregation work performed in the database.

Against the research questions, RQ1 is answered by Chapters IV and V together with Table **Error! Reference source not found.:** the three-tier Flask-React-PostgreSQL design and predominantly normalised data model satisfy the defined objectives within the evaluated scope. RQ2 is addressed by the generic, user-mapped importer, the representative statement-format design inputs and its five dedicated automated tests; the implementation avoids per-bank parser classes, although the evaluation does not claim compatibility with every future export variant. RQ3 is addressed by the JWT-plus-bcrypt design and the passing isolation test: within the selected individual self-hosted scope, signed tokens and per-user filtering provide the required authentication and data-isolation properties without a server-side session store.

VI.3.2 Connection to the Existing Literature

In relation to the literature reviewed in Chapter II, Vault belongs to the self-hosted web category represented by systems such as Firefly III (van Pelt, 2016), but its defaults and import workflow are directed at the Albanian context. Categories and budgets are consistent with the mental-accounting perspective described by Thaler (Thaler, 1999), while the effort to keep setup and interaction comparatively simple reflects the importance of perceived ease of use and effort expectancy in technology-adoption research (Davis, 1989), (Venkatesh et al., 2003). The separation of presentation, application and data responsibilities follows established layered-architecture guidance (Bass et al., 2021), (Fowler, 2002). Authentication follows the JWT

standard (Jones et al., 2015), with bcrypt password hashing (Provos & Mazières, 1999) and additional guidance from RFC 8725 and OWASP (OWASP Foundation, n.d.), (Jones et al., 2020)-(OWASP Foundation, n.d.). These connections support the design rationale but do not amount to a complete security or usability evaluation. Currency remains stored natively without automatic FX conversion, so mixed-currency reports must either preserve currency buckets or use a clearly defined reporting currency.

VI.3.3 Justifying the Major Design Decisions in Light of Results

The results also provide evidence for several design choices in Table **Error! Reference source not found.** Maintaining current_balance (Decision 1) avoids recomputing complete transaction histories for account and summary reads, while SQLAlchemy event handling (Bayer, 2006) keeps balance updates in a testable write path. The positive-amount convention (Decision 2) separates monetary magnitude from transaction direction. The generic importer (Decision 4) is exercised by format-focused tests and remains independent of bank-specific parser classes.

VI.4 Threats to Validity

VI.4.1 Internal Validity

The test procedure is reproducible from the repository: each case starts from a fresh SQLite database state, and the documented dependency versions are pinned or lock-resolved. Appendix C reports the exact environment used for the final native verification. The latency figures are less portable because they depend on hardware, platform, cache state and runtime versions; discarding initial requests and reporting medians reduces but does not eliminate this variability. The isolation test exercises API-level ownership checks rather than providing a penetration test. The evaluation also does not cover token theft, XSS, dependency vulnerabilities, secret management, TLS configuration or server-side token revocation.

VI.4.2 External Validity

Alternative technology stacks could plausibly satisfy similar requirements, but they were not built or evaluated here. The thesis therefore demonstrates one defensible point in the design space rather than a uniquely optimal architecture. The performance measurements are also limited to the demonstration dataset of roughly 150 transactions. Behaviour over years of use, tens of thousands of rows or high concurrency was not measured and remains an external-validity limitation.

VI.4.3 Construct Validity

The traceability matrix maps objectives to implemented features, but the breadth of each feature is established by the surrounding implementation evidence rather than by the matrix alone. The latency metric measures server-side response time; it does not include network delay, browser rendering or human interaction, and therefore should be interpreted as performance-efficiency evidence rather than a complete measure of perceived usability (ISO/IEC, 2023).

VI.4.4 Reliability

Repeatability is supported by deterministic tests, a fixed pseudo-random seed for demonstration data, and pinned or lock-resolved dependencies in requirements.txt and package-lock.json. A reader can rebuild the documented environment, regenerate the demonstration dataset for a given date and repeat the same procedures. Exact response times may still differ across hardware, operating systems and runtime versions.

Chapter VII: Conclusions and Future Work

VII.1 Conclusions

The work began from a practical limitation in the reviewed PFM landscape: none of the examined systems combined self-hosting, multi-currency support, an Albanian-context orientation and a generic local-bank CSV workflow with relatively low operational complexity. The comparison of Mint, YNAB, Firefly III and GnuCash in Chapter II was used to define this gap more precisely.

The resulting system consists of a React 18 frontend, a Flask 3 REST API and a PostgreSQL 15 database. When application code, tests and migrations are counted, the backend contains roughly four thousand lines of Python, while the frontend contains roughly three thousand lines of JavaScript, JSX and CSS. The source is published in a repository that includes Docker Compose configuration for reproducible deployment.

The evaluation shows that the artifact meets the defined objectives within the tested scope. The five canonical PFM functions are covered together with savings goals and CSV import. The ordinary interactive API endpoints measured below the one-hundred-millisecond reference on the demonstration dataset, while login and CSV import were slower. The sixteen-case automated suite passes in full, including the cross-user isolation test and five dedicated importer tests. The importer was designed against statement-export samples from BKT, Raiffeisen Bank Albania, Credins Bank and Tirana Bank and uses a generic mapping/parsing pipeline rather than bank-specific parser classes. These results support feasibility at demonstration scale rather than general scalability or complete security assurance. Within the evaluated self-hosted scope, JWT authentication, password hashing and enforced per-user filtering provided the required authentication and data-isolation properties; broader assurance would require penetration testing, dependency review, secure deployment configuration and adversarial testing.

The project makes both practical and academic contributions. Practically, it provides an open-source PFM application with lek as the default currency, locally selected categories and a CSV workflow designed against Albanian bank statement formats. Within the documentation reviewed for this thesis, no directly comparable open-source system with the same stated orientation was identified. Academically, the work records the path from problem definition to an implemented and measured software artifact, including the technical trade-offs and the limits of the evaluation.

VII.2 Future Work

The limitations identified in Section I.5.2 define several directions for future development and evaluation.

Direct bank API integration. Albania's open-banking framework under Law No. 55/2020 (Republic of Albania, 2020) is established and the Bank of Albania granted the country's first open-banking licence in November 2024 (Bank of Albania, 2024), with subsequent implementation context documented by the central bank (Bank of Albania, n.d.). Integration through an appropriately licensed provider could remove the CSV step for supported institutions. Because import and persistence concerns are separated in the current architecture, such integration could be added without redesigning the rest of the transaction model.

Exchange-rate integration. A daily Bank of Albania or European Central Bank reference-rate feed could support consolidated net-worth reporting while preserving the original currency recorded for each account and transaction.

Receipt photos and OCR. Capturing a receipt image and extracting its date, merchant and amount could reduce manual-entry effort and provide an attached source record for later review.

Investment tracking. The current schema includes an investment account type, but the reporting layer treats it as a standard balance. A fuller implementation would require market prices, cost basis, realised and unrealised gains, and appropriate historical valuation logic.

Anomaly detection. With sufficient history, statistical or machine-learning methods could flag unusual spending patterns. Such a feature would require careful threshold design and evaluation to avoid excessive false positives.

Shared budgets. Household use would require shared resources, role or permission rules, and an invitation mechanism. These additions would extend the ownership model beyond the individual-user scope evaluated in this thesis.

Native mobile clients. A React Native or Flutter client could reuse the existing REST API. An offline-first design would add further requirements for local storage, synchronisation, conflict resolution and consistency.

Albanian localisation. The interface is currently English, while the data model and defaults are oriented toward Albanian usage. Full interface translation is therefore future work. Existing formatting utilities use locale-aware browser facilities where applicable, but complete linguistic localisation has not been evaluated.

Bibliography

- Kaye, J. J., McCuiston, M., Gulotta, R., & Shamma, D. A. (2014). Money talks: Tracking personal finances. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14) (pp. 521-530). ACM.
<https://doi.org/10.1145/2556288.2556975>
- Bloomberg News. (2023, November 1). Intuit is closing personal-finance app Mint, shifts users to Credit Karma. Bloomberg. <https://www.bloomberg.com/news/articles/2023-11-01/intuit-winds-down-personal-finance-app-mint-shifts-users-to-credit-karma>
- Mandell, L., & Klein, L. S. (2009). The impact of financial literacy education on subsequent financial behavior. *Journal of Financial Counseling and Planning*, 20(1), 15-24.
- Intuit Inc. (2009). Intuit completes acquisition of Mint.com [Press release].
<https://investors.intuit.com/news-events/press-releases/detail/1005/intuit-completes-acquisition-of-mint-com>
- van Pelt, J. (2016-2026). Firefly III: A self-hosted free personal finance manager.
<https://www.firefly-iii.org/>
- European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 1-88.
- European Parliament and Council of the European Union. (2015). Directive (EU) 2015/2366 on payment services in the internal market (PSD2). *Official Journal of the European Union*, L 337, 35-127.
- Garman, E. T., & Forgue, R. E. (2018). *Personal finance* (13th ed.). Cengage Learning.
- The Verge. (2023, November 2). Mint is shutting down, and it's pushing users toward Credit Karma. <https://www.theverge.com/2023/11/2/23943254/mint-intuit-shutting-down-credit-karma>
- Eckerson, W. W. (1995). Three tier client/server architecture: Achieving scalability, performance, and efficiency in client/server applications. *Open Information Systems*, 10(1), 3-22.
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference (FREENIX Track).
- OWASP Foundation. (n.d.). Password storage cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Foundation. (2021). OWASP Top 10 web application security risks.
<https://owasp.org/Top10/>
- Republic of Albania. (2020). Law No. 55/2020 “On payment services”. Bank of Albania.
https://www.bankofalbania.org/rc/doc/Ligji_Per_sherbimet_e_pagesave_anglisht_18199.pdf [Online]. Available:
https://www.bankofalbania.org/rc/doc/Ligji_Per_sherbimet_e_pagesave_anglisht_18199.pdf.
- YNAB LLC. (n.d.). Direct import in Europe. YNAB Support.
https://support.ynab.com/en_us/direct-import-in-europe-Syae1z_A9
- GnuCash Project. (1998-2024). GnuCash: Free accounting software.
<https://www.gnucash.org/>
- Pacioli, L. (1494). Summa de Arithmetica, Geometria, Proportioni et Proportionalita.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75-105. <https://doi.org/10.2307/25148625>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45-77.
- Grinberg, M. (2018). Flask web development: Developing web applications with Python (2nd ed.). O'Reilly Media.
- Gunicorn Project. (2010-2024). Gunicorn: Python WSGI HTTP server for UNIX.
<https://gunicorn.org/>
- Nielsen, J. (1993). Usability engineering. Morgan Kaufmann.
- Banks, A., & Porcello, E. (2020). Learning React: Modern patterns for developing React apps (2nd ed.). O'Reilly Media.
- PostgreSQL Global Development Group. (2024). PostgreSQL 15 documentation.
<https://www.postgresql.org/docs/15/>
- Bayer, M. (2006-2024). SQLAlchemy: The database toolkit for Python.
- Lusardi, A., & Mitchell, O. S. (2014). The economic importance of financial literacy: Theory and evidence. Journal of Economic Literature, 52(1), 5-44.
<https://doi.org/10.1257/jel.52.1.5>

- Campbell, J. Y. (2006). Household finance. *The Journal of Finance*, 61(4), 1553-1604.
<https://doi.org/10.1111/j.1540-6261.2006.00883.x>
- Thaler, R. H. (1999). Mental accounting matters. *Journal of Behavioral Decision Making*, 12(3), 183-206.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319-340.
<https://doi.org/10.2307/249008>
- Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3), 425-478.
<https://doi.org/10.2307/30036540>
- (World Bank, 2022) World Bank. (2022). The Global Findex Database 2021: Financial inclusion, digital payments, and resilience in the age of COVID-19. World Bank.
- (OECD, 2020) OECD. (2020). OECD/INFE 2020 International Survey of Adult Financial Literacy. OECD.
- (Bass et al., 2021) Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
- (Fowler, 2002) Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- (Kleppmann, 2017) Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.
- Jones, M., Hardt, D., & Jones, M. B. (2020). JSON Web Token best current practices (RFC 8725). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8725>
- OWASP Foundation. (n.d.). Authentication cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Foundation. (n.d.). REST security cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- (Temoshok et al., 2025) Temoshok, D., et al. (2025). Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B-4). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63B-4> [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-63B-4>. [Accessed: 16 Aug. 2026].
- Bank of Albania. (2024). Open banking: Bank of Albania grants the licence to the first financial entity [Press release].
https://www.bankofalbania.org/Press/Press_Releases/Open_banking_Bank_of_Albania_grants_the_licence_to_the_first_financial_entity.html

- Bank of Albania. (n.d.). Open banking implementation in Albania.
https://www.bankofalbania.org/rc/doc/3_Open_Banking_Implementation_in_Albania_32369.pdf
- ISO/IEC. (2023). ISO/IEC 25010:2023 - Systems and software quality requirements and evaluation (SQuaRE) - Product quality model. International Organization for Standardization.
- (Ammann & Offutt, 2016) Ammann, P., & Offutt, J. (2016). Introduction to software testing (2nd ed.). Cambridge University Press.
- (Meszaros, 2007) Meszaros, G. (2007). xUnit test patterns: Refactoring test code. Addison-Wesley.
- (Elmasri & Navathe, 2016) Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Appendix A: Source Code Repository

The full source code accompanies this thesis in its Git repository:

1. backend/ - Flask backend application, tests and migration code (about 4,100 lines of Python in the reviewed repository; backend/app/ contains 28 application modules)
2. frontend/ - React single-page application source (about 3,200 lines of JSX, JavaScript, and CSS across 22 modules)
3. docs/ - System architecture diagram (docs/diagrams/architecture.svg) and entity-relationship diagram (docs/diagrams/er-diagram.svg)
4. docker-compose.yml - Multi-service deployment configuration (PostgreSQL, backend, frontend)
5. backend/tests/ - pytest test suite (16 cases across test_auth.py, test_transactions.py and test_imports.py)
6. backend/migrations/versions/ - Alembic database migration history (initial schema migration a1b2c3d4e5f6_initial_schema.py)
7. README.md - Installation and operation guide

The complete source is provided through the repository rather than reproduced in the body of the thesis. Chapter V includes only abbreviated excerpts, while Appendix A identifies the repository structure and Appendix D gives the installation procedure.

Repository link: <https://github.com/Krisjor/Vault-Finance-Tracker>

The repository can be cloned and run with the Docker Compose procedure documented in Appendix D.

Appendix B: Demonstration Dataset

The seeded dataset from Section III.2.2 is regenerated with:

```
docker-compose exec backend flask --app run seed-demo
```

The seed is fixed, so a given day always produces the same data: one user, four accounts (BKT Checking, Raiffeisen Savings and Cash Wallet in lek, plus a euro Revolut card), sixteen categories (twelve expense, four income), around 150 transactions over the preceding ninety days, five monthly budgets, and three savings goals.

Demonstration credentials after seeding:

```
Email:      demo@example.com  
Password: password123
```

Appendix C: Detailed Test Results

The reproducible Docker command is `docker-compose exec backend pytest -v`. The final verification recorded below was run natively from the backend directory with:

```
python -m pytest -v
```

Final native pytest output (machine-specific path and repeated warning details elided; warnings were deprecation notices only):

```
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: <project>\finance-tracker\backend
plugins: flask-1.3.0
collected 16 items

tests/test_auth.py::test_register_creates_user_and_returns_tokens PASSED
tests/test_auth.py::test_register_rejects_short_password PASSED
tests/test_auth.py::test_register_rejects_duplicate_email PASSED
tests/test_auth.py::test_login_succeeds_with_correct_credentials PASSED
tests/test_auth.py::test_login_rejects_wrong_password PASSED
tests/test_auth.py::test_me_requires_auth PASSED
tests/test_auth.py::test_me_returns_user_when_authenticated PASSED
tests/test_imports.py::test_csv_preview_comma_delimited PASSED
tests/test_imports.py::test_csv_preview_semicolon_delimited PASSED
tests/test_imports.py::test_csv_date_parsing_formats PASSED
tests/test_imports.py::test_csv_amount_parsing_locales PASSED
tests/test_imports.py::test_csv_import_sha256_deduplication PASSED
tests/test_transactions.py::test_create_expense_decreases_balance PASSED
tests/test_transactions.py::test_create_income_increases_balance PASSED
tests/test_transactions.py::test_delete_transaction_restores_balance PASSED
tests/test_transactions.py::test_user_cannot_see_other_users_transactions PASSED
===== 16 passed, 307 warnings in 6.79s =====
```

Appendix D: Installation Guide

Prerequisites:

1. Docker 24+ and Docker Compose 2.x (alternatively: Python 3.11.x, Node.js 20+, and PostgreSQL 15+ installed natively)
2. Git for source-code retrieval
3. About 2 GB of free disk space

Security note. The repository's Docker Compose secrets and database password are development placeholders. Before any non-demo deployment, replace SECRET_KEY, JWT_SECRET_KEY and database credentials, and place the application behind HTTPS/TLS (for example, through a reverse proxy).

Quick-start installation (Docker-based, recommended):

```
# 1. Clone the repository
git clone https://github.com/Krisjor/Vault-Finance-Tracker finance-tracker
cd finance-tracker

# 2. Bring up the three-service stack
docker-compose up -d --build

# 3. Initialize the database schema
docker-compose exec backend flask --app run db upgrade

# 4. Seed the demonstration dataset
docker-compose exec backend flask --app run seed-demo

# 5. Open the web application
# Frontend: http://localhost:5173
# Backend: http://localhost:5000
```